

2023年

情報科学実験 I 指導書



A コンテンツ制作と音声処理



B 画像処理と生成



C 通信ネットワーク



D データ科学と自動制御



愛知県立大学 情報科学部

Aichi Prefectural University

目次

0. はじめに

0-1 実験・目的

0-2 予習

0-3 レポート

1. 第1週：JAVA GUI プログラミングによるエージェント・コンテンツ生成

1-1 GUI プログラミングとは

1-2 GUI ライブラリ

1-3 SWING の基礎操作

1-3-1 ウィンドウ

1-3-2 ボタンとラベル

1-3-3 アクションリスナー

1-3-4 写真表示

1-3-5 レイアウト

1-4 SWING の応用操作

1-4-1 音声再生

1-4-2 ラジオボタン

1-4-3 画面切り替え

1-5 TexTalk の使い方

2. 第2週：実験デザイン／アンケートの作成・選定

2-1 被験者実験

2-2 アンケート方法

2-2-1 SD 法

2-2-2 GodSpeed

2-2-3 五段階評価尺度法(リッカート尺度)

2-3 アンケートの作成・選定

3. 第3週：検定／分析方法

3-1 検定

3-3-1 検定とは

3-3-2 検定の流れ

3-3-3 母分散が未知の場合の母平均の検定

参考文献

0. はじめに

0-1 実験・目的

本実験の目的は、JAVA・GUI プログラミング SWING を理解して、GUI プログラミングを用いて画面エージェントのコンテンツを各自で開発します(第1週)。また、開発したコンテンツを被験者実験によるアンケートを基に評価します(第2週)。そして、評価結果に統計的な検定を実施して、開発したコンテンツの良い点・悪い点を考察します(第3週)。

0-2 準備・予習

実験を効率的に実施するため、事前に第1・2・3週の内容をよく読み、実験内容を理解しておくこと。また、課題についても、しっかり読み、内容を十分に把握すること。特に第1週では JAVA プログラミングを使用します。演習室では実行環境が整っていますが、自身のノートパソコンで実行したい場合は、各自で実行環境を構築してください。音声ファイルも使用するのでイヤホンなども実験当日に持ってきてください。また、事前に第1週の JAVA プログラミングを実行して、内容を確認しておくと実験がスムーズに進行できます。

0-3 レポート

レポートは、第三者が各実験の課題や内容を正確に伝わるように執筆してください。第1週のレポートでは、コンテンツの目的、対象者、アピール点を明確に説明してください。第2週のレポートでは、アンケートから得られた結果のみで、コンテンツを評価し、まとめてください。第3週のレポートでは、アンケート結果に検定を実施し、検定結果のみでコンテンツを評価し、第2週のレポートと比較してください。

第1週

- 開発したコンテンツの目的が明確に記載しているか
- コンテンツの対象者および与える影響は説明できているか

第2週

- アンケートの作成・選定の理由を明確に説明できているか
- 評価結果から得られた事実のみで、考察できているか

第3週

- 検定結果からコンテンツを客観的に評価できているか
- 第2週のレポートと検定結果による考察を比較できているか

第1週：JAVA GUI プログラミングによるエージェント・コンテンツ生成

1-1 GUI プログラミングとは

GUI は、グラフィカル・ユーザ・インタフェース(Graphical User Interface)の略語であり、グラフィカルな画面を指しています。マウスやタッチパネルで操作が出来る画面であれば、GUI として扱えます。つまり、デスクトップパソコン、スマートフォンやカーナビ、ATM の画面なども全て GUI として扱えます。

GUI の反対は、CUI となります。CUI は、キャラクターベース・ユーザ・インタフェース(Character-based User Interface)の略語であり、文字ベースのユーザ・インタフェースです。Windows のパソコンであれば、コマンドプロンプトが該当します。コマンドプロンプトは、マウスやタッチパネルで操作することができません。つまり、何かしらの操作を行いたい場合は必ずそれ専用の命令を文字で記述する必要があります。

GUI プログラミングとは、GUI を構築できるプログラミングを指しています。GUI プログラミングには、Visual C++、JAVA、Python など多種多様な種類があります。本学では、プログラミング演習で JAVA を扱っていますが、演習では CUI の部分を中心に組みました。本実験では、JAVA における GUI プログラミングについて学んでいきます。



図1 GUI の画面例(デスクトップ PC)



図2 CUI の画面例(コマンドプロンプト)

1-2 GUI ライブラリ

JAVA の GUI プログラミングにおけるライブラリ(パッケージ)として、さまざまな種類があります。通常、JAVA の GUI プログラミングで下記のライブラリを使用します。

◆ awt

基本的なユーザ・インタフェースの作成およびグラフィックスとイメージの描画用のすべてのクラスを含むパッケージです。GUI コンポーネントとして awt を使った場合、デザインが環境に依存する(実行する環境によって異なった外観になる)という問題が発生することがあります。しかし、awt の基本的なユーザ・インタフェースについては、他のパッケージを併用したプログラミングでも問題なく使用できます。

◆ Swing

すべてのプラットフォーム上で可能なかぎり同じように機能する一連の「軽量」コンポーネント(システムの構成要素)として使用します。以前は Swing は実行時の速度が遅いとされていたが、さまざまな面で改良や改善が行われて、現在では実用的な速度が実現されている。本実験では、Swing を基にコンテンツを開発する。

◆ SWT

Eclipse Foundation が Eclipse IDE と共に管理保守している Java 用 GUI 部品のツールキットのひとつです。開発用の統合ツールである Eclipse IDE と共に使用されることを前提としています。環境に依存する部分が多く、作成したプログラムが Java の動作するプラットフォームすべてで動作するとは限りません。またリソースをプログラムで必ず解放しなければならないなど、やや難しい面があります。

◆ JavaFX

グラフィックスやアニメーション、ビデオなどを容易に扱うことが可能になるように設計されたもので、XML と CSS を併用してデザインを記述します。Java 11 以降、JavaFX は Oracle JDK に含まれておらず、スタンドアロンライブラリとして利用するようになりました。視覚的要素が特に重要なアプリの開発に適しています。

1-3 SWING の基礎操作

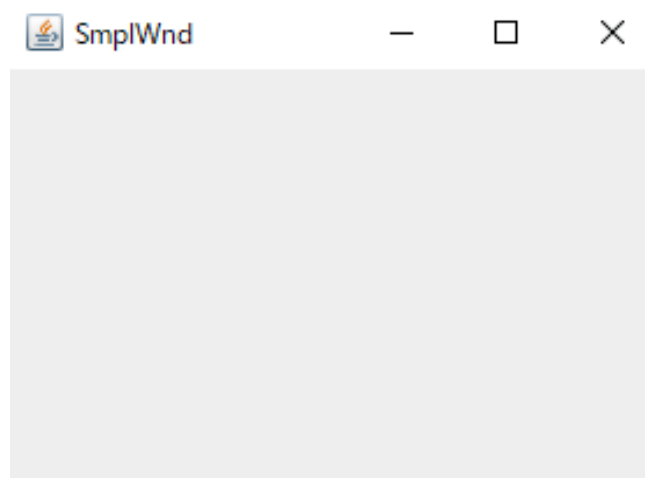
SWING における基礎的なプログラミングについて説明します。ウィンドウ、ボタン、ラベル、写真表示、レイアウトに関するプログラムコードを下記に示します。プログラムコードには、コードの意味を説明するコメントが記載されているので、確認しながらプログラムを実行してください。また、プログラムが実行された場合に、表示される画面(ウィンドウ)の写真も示します。

1-3-1 ウィンドウ

SmplWnd.java

```
import javax.swing.*;

public class SmplWnd extends JFrame {
    public static void main(String[] args) {
        new SmplWnd(); // SmplWnd()関数を呼び出します。
    }
    public SmplWnd(){
        this.setTitle("SmplWnd"); // ウィンドウにタイトルを付加。
        this.setSize(300, 220); // ウィンドウのサイズを定義。
        this.setVisible(true); // ウィンドウの表示を許可(true)
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        // ウィンドウの閉じたらプログラムを終了
    }
}
```

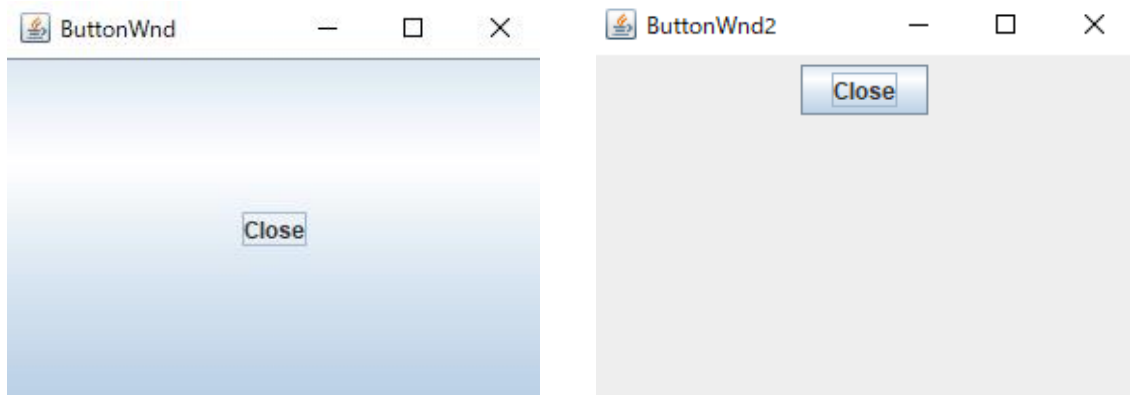


1-3-2 ボタンとラベル

ButtonWnd.java

```
import javax.swing.*;

public class ButtonWnd extends JFrame{
    public static void main(String[] args) {
        new ButtonWnd();
    }
    public ButtonWnd(){
        JButton button = new JButton("Close"); // 「Close」という名のボタンを作成
        this.add(button); // ボタンをウィンドウに追加
        this.setTitle("ButtonWnd");
        this.setSize(300, 220);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```



ボタンをウィンドウに追加した場合、ButtonWndのようにウィンドウ全体にボタンが追加されます。そのため、ボタンなどを使用する際は、透明なウィンドウであるパネル (Panel) に一度ボタンを追加した後に、ウィンドウへ追加します。これにより、ButtonWnd2のように小さいボタンを使用できます。

ButtonWnd2.java

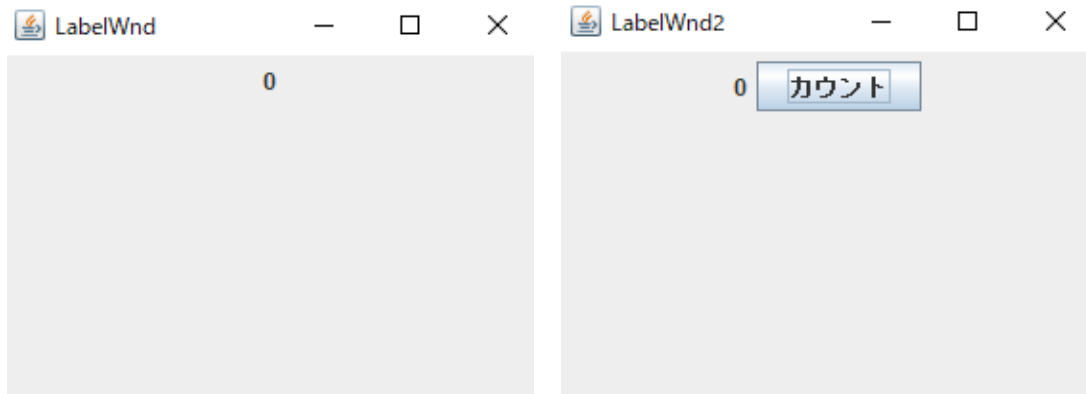
```
import javax.swing.*;

public class ButtonWnd2 extends JFrame{
    public static void main(String[] args) {
        new ButtonWnd2();
    }
    public ButtonWnd2(){
        JButton button = new JButton("Close");
        JPanel buttonPanel = new JPanel(); //透明なウィンドウのパネルを作成
        buttonPanel.add(button); //パネルにボタンを追加
        this.add(buttonPanel); //パネルをウィンドウに追加
        this.setTitle("ButtonWnd2");
        this.setSize(300, 220);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

LabelWnd.java

```
import javax.swing.*;

public class LabelWnd extends JFrame{
    public static void main(String[] args) {
        new LabelWnd();
    }
    public LabelWnd(){
        JLabel label = new JLabel("0"); //「0」というはり紙(ラベル)を作成
        JPanel labelPanel = new JPanel();
        labelPanel.add(label); //ラベルをパネルに追加
        this.add(labelPanel); //パネルをウィンドウに追加
        this.setTitle("LabelWnd");
        this.setSize(300, 220);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

ラベル(はり紙)は文字表示などに使用します。ラベルの特徴として、Html のプログラムコードを使用することがあります。つまり、文字色などを変更したい場合は、`JLabel("<html><font color='red'>0</font></html>")`と Html のプログラムコードを `Label()`内に記載することで、Html のように文字色などを変更できます。また、`LabelWnd2` のようにボタンと組み合わせて使用することもできます。

LabelWnd2.java

```
import java.awt.*;
import javax.swing.*;

public class LabelWnd2 extends JFrame{
    public static void main(String[] args) {
        new LabelWnd2();
    }
    public LabelWnd2(){
        JLabel label = new JLabel("0");
        JButton button = new JButton("カウント");
        JPanel panel = new JPanel();
        panel.add(label); //ラベルをパネルに追加
        panel.add(button); //ボタンをパネルに追加(ラベル→ボタンを表示)
        this.add(panel);
        this.setTitle("LabelWnd2");
        this.setSize(300, 220);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

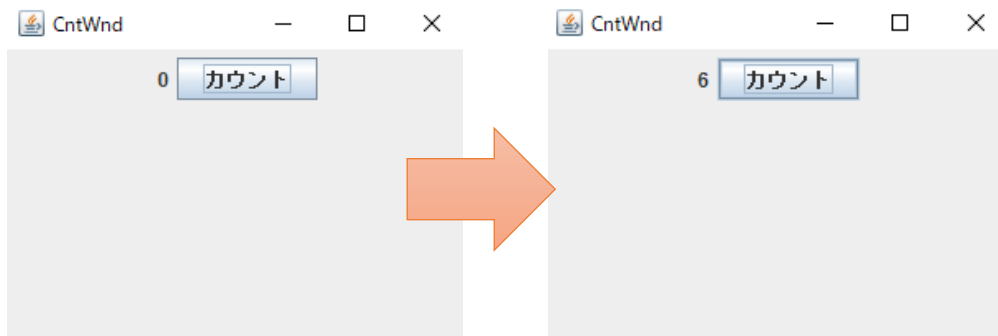
1-3-3 アクションリスナー

アクションリスナーとは、ボタンが押下されたなどの特定の動作が行われた場合に実行するプログラムのことを指します。

CntWnd.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

public class CntWnd extends JFrame{
    int count = 0; //ActionListener に使用するため、グローバル変数を作成
    JLabel label = new JLabel(""+count+"");
    //ラベル内は文字しか扱えないので、int 型変数を""と""で加算し、文字型へ変更
    // ActionListener に使用するため、グローバルラベルとして作成
    public static void main(String[] args) {
        new CntWnd();
    }
    public CntWnd(){
        JButton button = new JButton("カウント");
        button.addActionListener(new Counter()); //ボタンにアクション機能を追加
        JPanel panel = new JPanel();
        panel.add(label);
        panel.add(button);
        this.add(panel);
        this.setTitle("CntWnd");
        this.setSize(300, 220);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
    class Counter implements ActionListener{
        //ActionListener では、ボタンが押下されたときに実行されるプログラムを記載
        public void actionPerformed(ActionEvent evt){
            count++; //グローバル変数ではないと、変更や再定義で不可能
            label.setText(""+count+""); //label に()内の内容を再定義
        }
    }
}
```



「カウント」ボタンが押下されるたびに、actionPerformed 内のプログラムが実行されて、数字「0」が+1 ずつ加算されて、ウィンドウ内に表示されます。

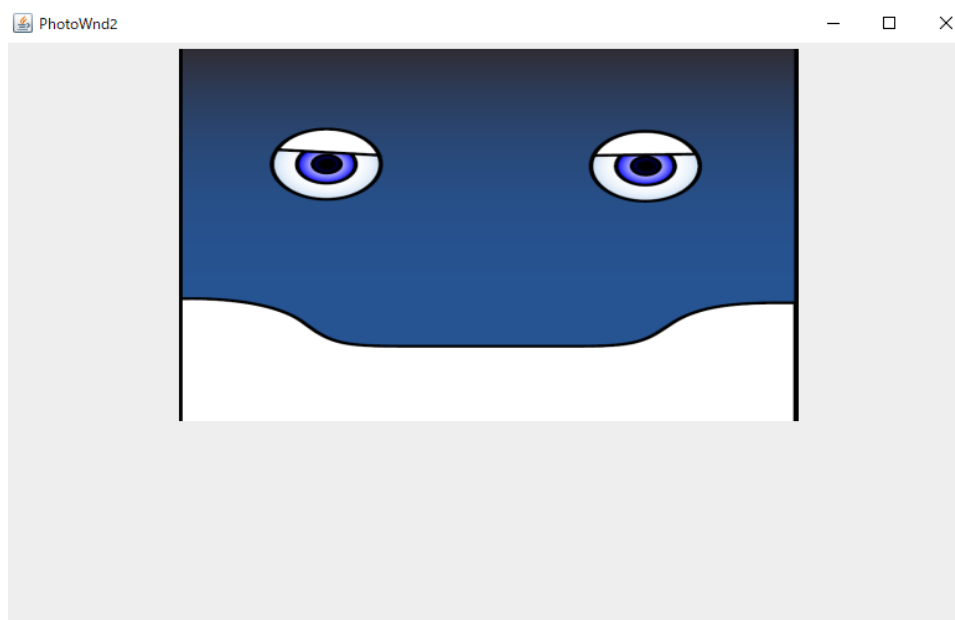
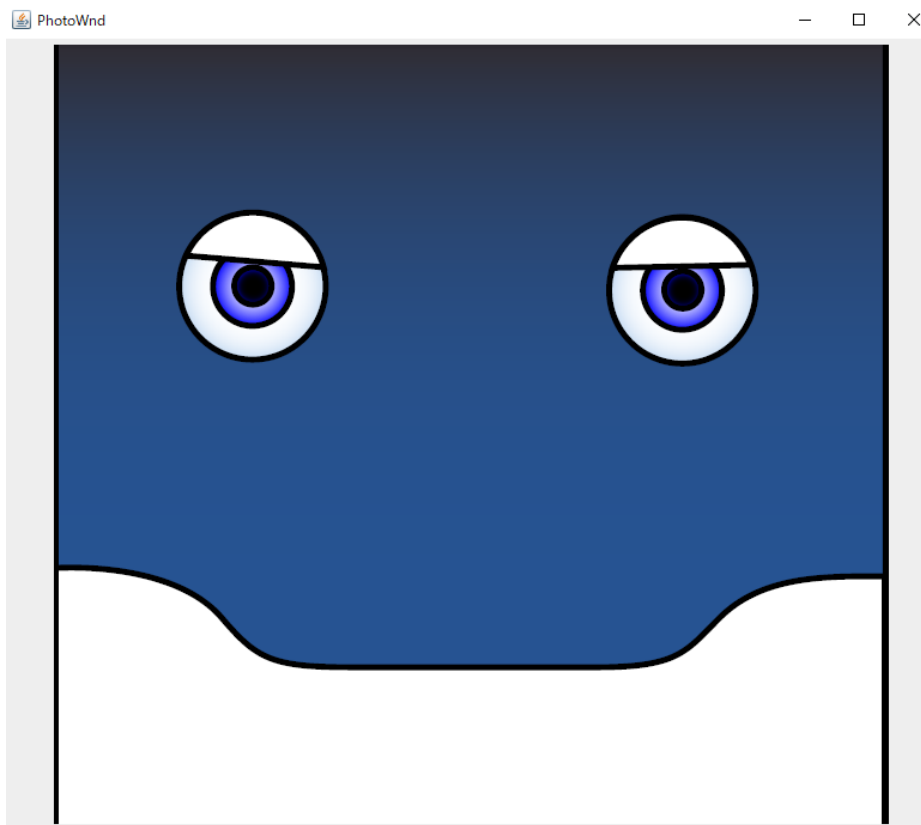
1-3-4 写真表示

PhotoWnd.java

```
import java.awt.*;
import javax.swing.*;

public class PhotoWnd extends JFrame{
    public static void main(String[] args) {
        new PhotoWnd();
    }
    public PhotoWnd(){
        ImageIcon Icon = new ImageIcon("標準.png"); //表示する写真を定義
        JLabel phote = new JLabel(Icon); //ラベルに写真を追加
        JPanel panel = new JPanel();
        panel.add(phote);
        this.add(panel);
        this.setTitle("PhotoWnd");
        this.setVisible(true);
        this.setSize(800, 700);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

写真を表示するには、ImageIcon で表示する写真を定義する必要があります。Icon は変数名であり、new ImageIcon() の()内で表示する写真(png, jpg, gif など)を定義します。



PhotoWnd2.java

```
import java.awt.*;
import javax.swing.*;

public class PhotoWnd2 extends JFrame{
    public static void main(String[] args) {
        new PhotoWnd2();
    }
    public PhotoWnd2(){
        ImageIcon Icon = new ImageIcon("標準.png");
        Image image0 = Icon.getImage(); //画像のサイズデータを取得
        Image newimg0
            = image0.getScaledInstance(500, 300, java.awt.Image.SCALE_SMOOTH);
            //幅 500, 高さ 300 の画像へ縮小(SMOOTH)

        Icon = new ImageIcon(newimg0);
        JLabel phote      = new JLabel(Icon);
        JPanel panel = new JPanel();
        panel.add(phote);
        this.add(panel);
        this.setTitle("PhotoWnd2");
        this.setVisible(true);
        this.setSize(800, 700);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

画像を縮小化するときは、画像の情報を `getImage()` で取得した後に、`getScaledInstance` を用いて、サイズを指定して縮小します。その際の注意点として、`image0` や `newimg0` のように変数を使い分けないと縮小化がうまく実行できないことがあります。また、縮小化できる画像は、png もしくは jpg のみです。gif などは縮小できません。

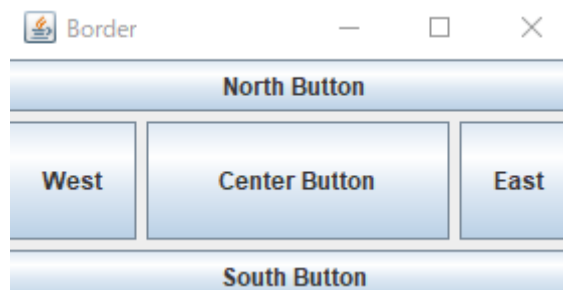
1-3-5 レイアウト

レイアウトは複数の写真やボタンを綺麗に表示させたい場合に使用します。レイアウトの調整方法はいくつかありますが、ここでは基本的なものを紹介します。

Boder.java

```
import java.awt.*;
import javax.swing.*;

public class Border extends JFrame {
    public static void main(String[] args) {
        new Border();
    }
    public Border() {
        JButton btnn = new JButton("North Button");
        JButton btns = new JButton("South Button");
        JButton btnw = new JButton("West");
        JButton btne = new JButton("East");
        JButton btnc = new JButton("Center Button");
        this.setLayout( new BorderLayout( 5, 5 ) );
        //ウィンドウのレイアウトを定義, ()内には各配置場所間の縦横の間隔幅を定義
        this.add(btnn, BorderLayout.NORTH); //ウィンドウの NORTH(北・上)に配置
        this.add(btns, BorderLayout.SOUTH); //ウィンドウの SOUTH(南・下)に配置
        this.add(btnw, BorderLayout.WEST); //ウィンドウの WEST(西・左)に配置
        this.add(btne, BorderLayout.EAST); //ウィンドウの EAST(東・右)に配置
        this.add(btnc, BorderLayout.CENTER); //ウィンドウの CENTER(中央)に配置
        this.setTitle("Border");
        this.setSize(300, 160);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```



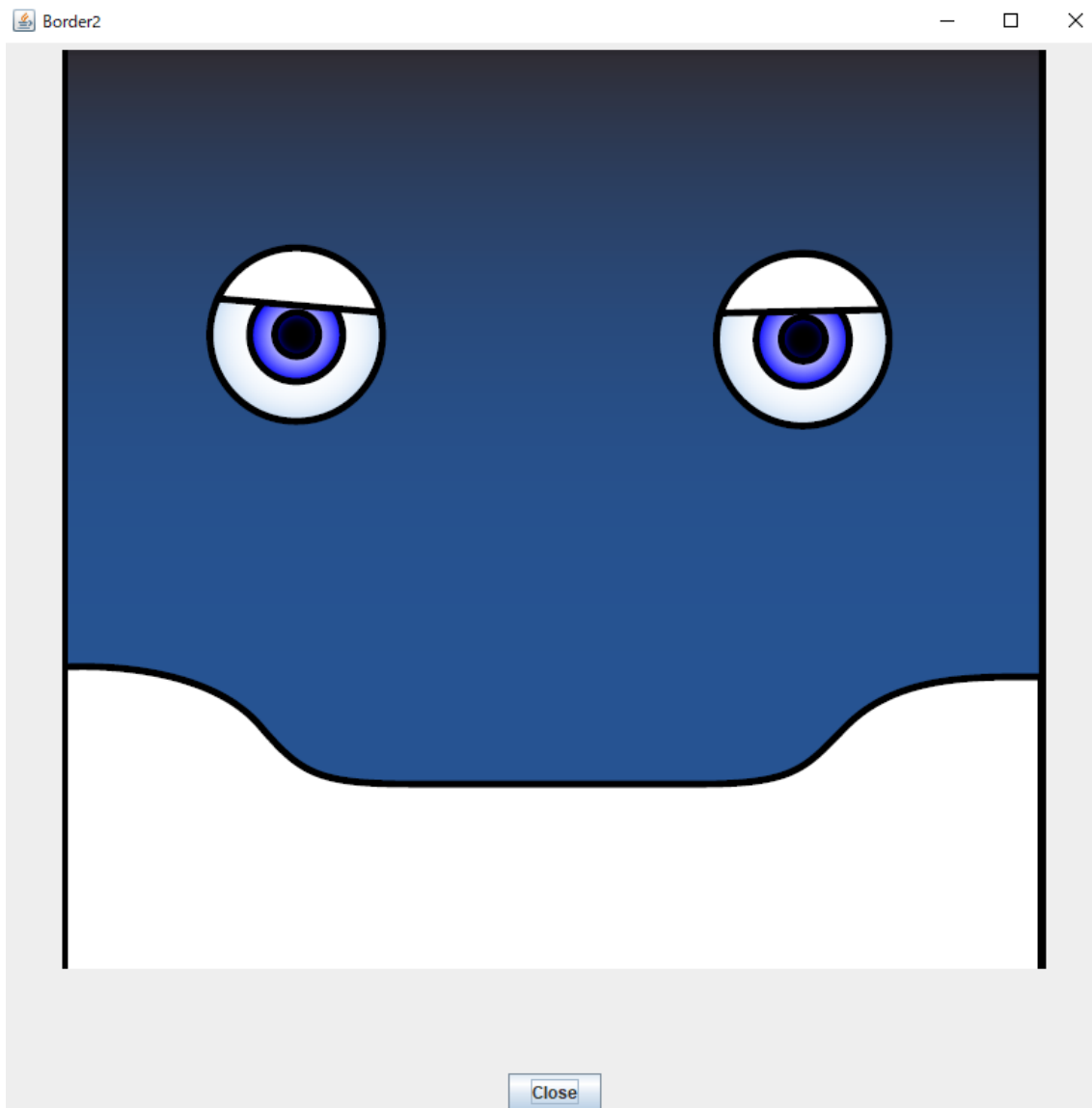
Boder2.java

```
import java.awt.*;
import javax.swing.*;

public class Border2 extends JFrame {
    public static void main(String[] args) {
        new Border2();
    }

    public Border2() {
        ImageIcon Icon    = new ImageIcon("標準.png");
        JLabel photo      = new JLabel(Icon);
        JPanel panel = new JPanel(); //画像用のパネルを作成
        panel.add(photo);
        JButton button = new JButton("Close");
        JPanel buttonPanel = new JPanel(); //ボタン用のパネルを作成
        buttonPanel.add(button);
        this.setLayout( new BorderLayout( 5, 5) );
        this.add(panel, BorderLayout.NORTH);
        //画像用パネルをウィンドウの北部分に追加／配置
        this.add(buttonPanel, BorderLayout.SOUTH);
        //ボタン用パネルをウィンドウの南部分に追加／配置
        this.setTitle("Border2");
        this.setSize(800, 800);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

Borber2 では、画像とボタンを二つ表示させています。課題にも使用しますが、Borber2.java のように、画像用とボタン用のパネルを用意します。そして、それぞれに対してウィンドウでの配置場所を指示します。



課題では，上記のようなエージェント・コンテンツを作ってもらいます．課題欄に詳しく説明していますが，画像(エージェント)を表示させ，コンテンツに応じて，下部分に，追加画像を表示させることや，音声を再生することなどを行います．

Box.java

```
import java.awt.*;
import javax.swing.*;

public class Box extends JFrame {
    public static void main(String[] args) {
        new Box();
    }

    public Box() {
        JPanel panel = new JPanel();
        panel.setLayout( new BoxLayout( panel, BoxLayout.Y_AXIS) );
        //パネルのレイアウトを定義, Y_AXIS により縦で配置することを定義
        JButton btn[] = new JButton[5];
        for (int i=0; i<5; i++){
            btn[i] = new JButton("Button:" + Integer.toString(i));
            btn[i].setAlignmentX(0.5f); //画面上の位置を指示, 0.5f により中央を指定
        }
        for (int i=0; i<5; i++){
            panel.add(btn[i]);
            JLabel BRLabel = new JLabel("<HTML><BR></HTML>");
            panel.add(BRLabel);
        }
        this.add(panel);
        this.setTitle("Box");
        this.setSize(220, 250);
        this.setVisible(true);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}
```

Border レイアウトでは、北部分に一度配置すると、さらに北部分に何かを配置した場合、上書きされてしまいます。そのため、パネルに対してレイアウトを指示した後に、ウィンドウでレイアウトを指示すると良いです。Box レイアウトは、Y_AXIS(縦)と X_AXIS(横)を指示することで、縦順や横順にボタンなどを並べることができます。



1-4 SWING の応用操作

1-4-1 音声再生

FaceSpeak.java

```
import java.io.*;
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.sound.sampled.*;

public class FaceSpeak extends JFrame{
    public static void main(String[] args) {
        new FaceSpeak();
    }
    public FaceSpeak(){
        ImageIcon Icon    = new ImageIcon("標準.png");
        JLabel phote      = new JLabel(Icon);
        JPanel panel = new JPanel();
        panel.add(phote);
        this.add(panel);
        this.setTitle("FaceSpeak");
        this.setVisible(true);
        this.setSize(800, 700);
        this.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        PlayVoice("あいさつ.wav");//PalyVoice に再生する音声ファイル(wav)を送信
```

```

    }
    public void PlayVoice(String wav){
        final File file = new File(wav);
        new Thread(new Runnable(){
            public void run(){
                System.out.println("Start...");
                try {
                    AudioInputStream in =AudioSystem.getAudioInputStream(file);
                    AudioFormat format = in.getFormat();
                    DataLine.Info info = new DataLine.Info(Clip.class, format);
                    Clip clip = (Clip)AudioSystem.getLine(info);
                    clip.open(in);
                    clip.start();
                } catch (UnsupportedAudioFileException e) {
                    e.printStackTrace();
                } catch (IOException e) {
                    e.printStackTrace();
                } catch (LineUnavailableException e) {
                    // TODO 自動生成された catch ブロック
                    e.printStackTrace();
                }
            }
        }).start();
    }
}

```

音声再生には、PlayVoice()関数をそのまま使用してください。PlayVoice 関数では、スレッドを用いることで、JFrame のプログラムと音声再生するプログラム(new Thread 内のプログラム)を並列実行しています。FaceSpeak.java を実行すると PhotoWnd.java と同じウィンドウが表示され、音声ファイルも再生されます。

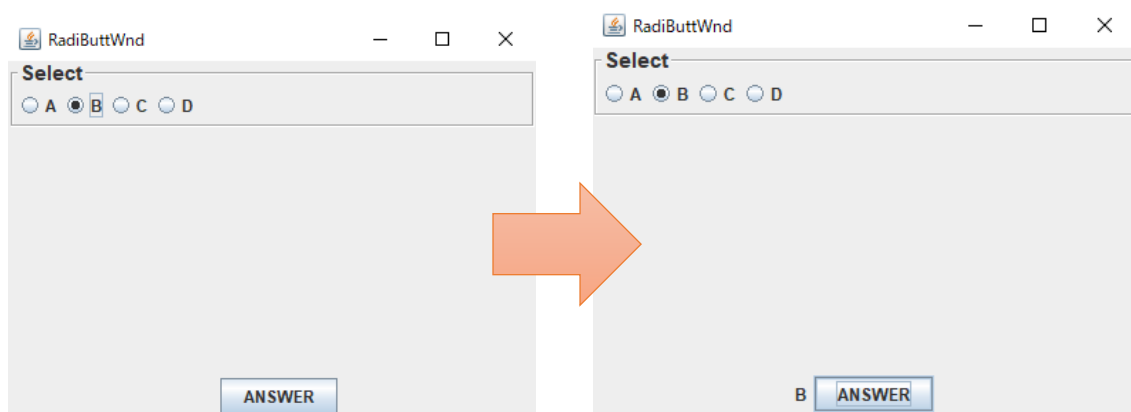
1-4-2 ラジオボタン

RadiButtWnd.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;
import javax.swing.border.*;

public class RadiButtWnd extends JFrame {
    RadioButtonPanel radipanel = null; // ActionListener に使用するためグローバル変数
    JLabel label = new JLabel(""); // ActionListener に使用するためグローバル変数
    public static void main(String[] args) {
        new RadiButtWnd();
    }
    public RadiButtWnd(){
        radipanel = new RadioButtonPanel("Select", new String[]{"A","B","C","D"});
        // 「Select」という題目がついたラジオボタンを生成
        // ラジオボタンの選択肢は、「A」、「B」、「C」、「D」と定義
        JButton button = new JButton("ANSWER");
        button.addActionListener(new SelectedAnswer());
        JPanel buttonPanel = new JPanel();
        buttonPanel.add(label);
        buttonPanel.add(button);
        this.add(radipanel, BorderLayout.NORTH);
        this.add(buttonPanel, BorderLayout.SOUTH);
        this.setTitle("RadiButtWnd");
        this.setSize(400, 300);
        this.setVisible(true);
    }
    class SelectedAnswer implements ActionListener{
        public void actionPerformed(ActionEvent evt){
            String answer = radipanel.getSelection();
            // ラジオボタンで選択されたものを取得
            label.setText(answer); // 取得情報をラベルに追加
        }
    }
}
```

```
//ラジオボタンを作成するプログラム， 使用する場合はそのまま使用すること
class RadioButtonPanel extends JPanel{
    public RadioButtonPanel(String title, String[] options){
        setBorder(BorderFactory.createTitledBorder
            (BorderFactory.createEtchedBorder(),title,TitledBorder.LEFT,
                TitledBorder.TOP,new Font("Arial", Font.BOLD, 15)));
        //「Arial」フォントの「BOLD(太字)」で大きさ「15」と定義
        setLayout(new BoxLayout(this, BoxLayout.X_AXIS));
        //選択肢を X_AXIS(横順)に並べると定義， Y_AXIS(縦順)にも変更可能
        group = new ButtonGroup();
        for(int i=0; i<options.length; i++){
            JRadioButton b = new JRadioButton(options[i]);
            b.setActionCommand(options[i]);
            add(b);
            group.add(b);
            b.setSelected(i==0);
        }
    }
    public String getSelection(){
        return group.getSelection().getActionCommand();
    }
    private ButtonGroup group;
}
```



ラジオボタンの選択肢を一つ選んだ後に、「ANSWER」ボタンを押下すると，「ANSWER」ボタンの横に選択された文字が表示されます。再度，前と異なる選択肢を選び直し，「ANSWER」ボタンを押下すると，選び直した選択肢が表示されます。

1-4-3 画面切り替え

FaceChange.java

```
import java.awt.*;
import java.awt.event.*;
import javax.swing.*;

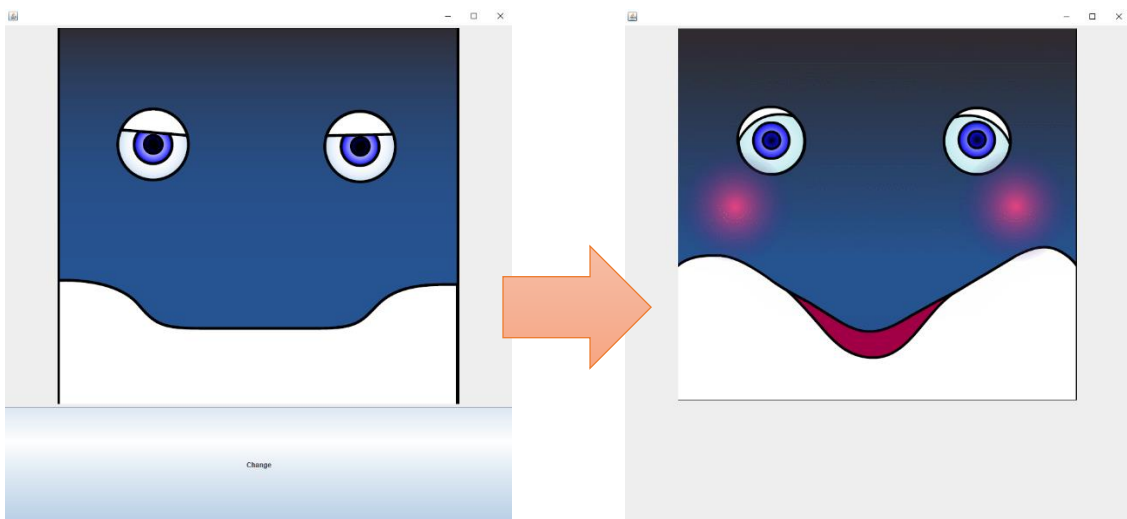
public class FaceChange extends JFrame{
    JPanel TabotPanel = null; //切り替えたいパネルをグローバル変数として定義
    JButton showButton = null; // ActionListener に使用するためグローバル変数
    public FaceChange(){
        this.setSize(900, 900);
        this.setVisible(true);
        show(0); //0 番目の画面を表示と指示(ウィンドウの詳細な定義は show)
    }
    public static void main(String[] args) {
        new FaceChange();
    }
    public void show(int num){
        switch(num){ //switch を使用することで複数枚の画面の切り替えを定義
            case 0: //0 番目のウィンドウ
                if(TabotPanel != null){
                    this.remove(TabotPanel); //TabotPanel をウィンドウから外す
                }
                TabotPanel = new JPanel();
                ImageIcon Icon = new ImageIcon("標準.png");
                JLabel phote = new JLabel(Icon);
                phote.setAlignmentX(0.5f); //写真を中央に設置と定義
                TabotPanel.add(phote);
                showButton = new JButton("Change");
                showButton.addActionListener(new ChangeA());
                this.add(TabotPanel, BorderLayout.NORTH);
                this.add(showButton, BorderLayout.CENTER);
                this.repaint(); //ウィンドウを再度作り直すと指示
                this.validate();
                break; //break することで 1 番目のウィンドウを表示させない
            case 1: //1 番目のウィンドウ
                if(TabotPanel != null){
```

```

        this.remove(TabotPanel); //TabotPanel をウィンドウから外す
    }
    if(showButton != null){
        this.remove(showButton); // showButton をウィンドウから外す
    } //Show(0)にあった Panel を外さないと二重で表示されることがあります.
    TabotPanel = new JPanel();
    ImageIcon Icon2    = new ImageIcon("喜び(弱).png");
    JLabel phote2      = new JLabel(Icon2);
    phote2.setAlignmentX(0.5f);
    TabotPanel.add(phote2);
    this.add(TabotPanel, BorderLayout.NORTH);
    this.repaint();
    this.validate();
    break;
    }
}

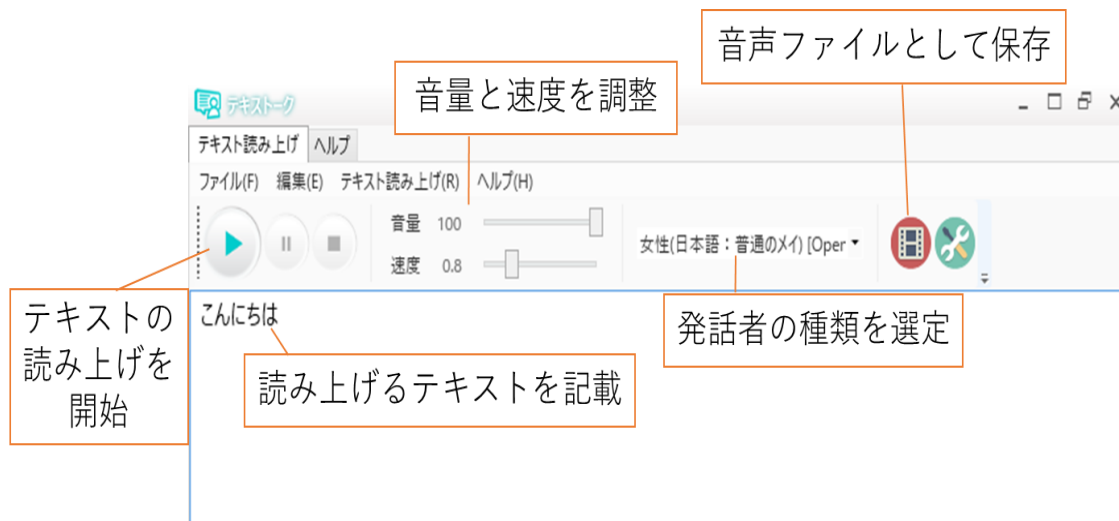
class ChangeA implements ActionListener{
    public void actionPerformed(ActionEvent evt){
        show(1); // showButton が押下された Show(1)を実行
    }
}
}

```



最初, ウィンドウは Show(0)で定義されたパネルやボタンが表示されます。「Change」ボタンを押下することで, Show(0)で定義されたパネルが表示されるようになります。

1-5 TexTalk の使い方



エージェントの音声作成には、「テキストトーク」というソフトウェアを使用します。ソフトウェアでは、日本語の文書を音声読み上げ(音声合成)することができます。そして、読み上げた音声を WAV 形式と MP3 形式で保存可能です。本実験において、WAV 形式の音声ファイルを再生できるプログラムコードとなっているので、音声は WAV 形式で保存してください。ソフトウェアの利用手順は下記に示します。

1. 「textalk.exe」を実行してテキストトークを起動します。
2. テキストボックスに読み上げるテキストを設定します。
3. ツールバーで音量/速度を設定し、声を選んでください。
4. ツールバーの再生ボタンを押し、テキスト読み上げを開始します。
(* 行毎に音声変換と再生をおこないます)
5. 全てのテキストの再生が終わると自動的に停止します。
途中で一時停止するには「一時停止」ボタンを押してください。
途中で再生を終了するには「停止」ボタンを押してください。
6. 音声を WAV 形式で保存してください。

第1週・課題

エージェント・コンテンツを開発しよう！！

目的

対象者に対して特定の情報を提供するエージェント・コンテンツを開発

評価基準

- 誰にも真似されないようなオリジナリティ／独自性があるか
- 提供する情報およびターゲットが明記されているか
- SWING プログラミングをうまく活用しているか

条件

1. 対象者と提供する情報をレポートに明記すること
2. JAVA・SWING プログラミングを使用すること
3. エージェント・コンテンツであるため、エージェントの表情は必ず表示させること
4. エージェントの表情は Teams 内のフォルダ「エージェント表情」を使用すること
5. インターネットから画像を取得した場合は、レポートに URL を必ず記載すること
6. 最低1回は音声を再生させること
7. 最低1回は画面の切り替えを実施すること

提出方法・べ切

- ✧ レポート(PDF)およびエージェントのプログラム・一式(ZIP)をメールにて提出
- ✧ ZIP ファイル名(PDF 名も)は「学籍番号」にすること
- ✧ 提出先：jimenez@cis.aichi-pu.ac.jp
- ✧ 件名：コンテンツ・第1週レポート
- ✧ 提出べ切：第2週目の実験日前日 16:00

注意点：

- ✓ 友人同士で話し合っても良いですが、必ず一人一つ開発
- ✓ 全く同じエージェント・コンテンツ(過去も含め)があった場合は、再提出
- ✓ 第2週までには、エージェント・コンテンツを開発させ、実行可能状態へ

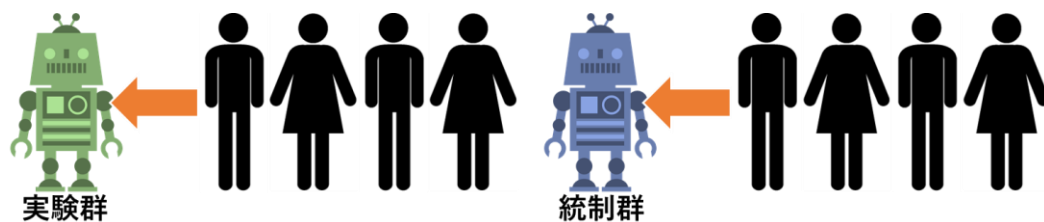
第2週：実験デザイン／アンケートの作成・選定

2-1 被験者実験

エージェント／ロボットやシステムを評価するには、実際に人にシステムを使用してもらうことや、人とエージェントやロボットが関わる(インタラクション)を行うことなどを被験者実験として実施します。被験者実験には、被験者間実験(被験者間配置)と被験者内実験(被験者内配置)の2種類の実施方法があります。検証したい内容に応じて実施方法を選択する必要があります。また、被験者実験は男女比も統一することが望ましいです。

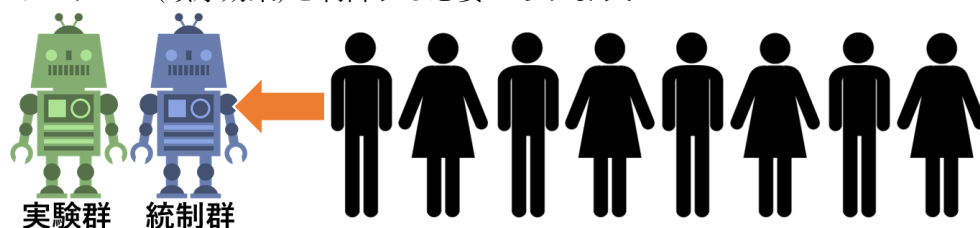
被験者間実験

被験者一人につき一つの実験条件を実施する方法です。例えば、新たに開発したエージェント／ロボットとインタラクションを行う「実験群」と、従来のエージェント／ロボットとインタラクションを行う「統制群」に対して、被験者を一人ずつ振り分けて実験を実施します。被験者ごとの個人差の影響を受けやすいため多くの被験者が必要となる傾向があります。そのため、十分に多くデータを集める必要であり、多くのデータを集めることで正確なデータを得ることができます。



被験者内実験

被験者一人ごとに全ての実験条件を実施する方法です。例えば、一人の被験者が実験群と統制群における両方のロボットとインタラクションを行う実験を実施します。被験者の個人差の影響はほとんど生じません。しかしながら、一度実施した実験条件の影響が以後に繰り越すといった繰越し効果が発生します。具体的には、実験群のロボットの後に、統制群のロボットとインタラクションを行った被験者は、実験群の経験から初回とは行った行動、反応、評価を行われると考えられます。そのため、実施順をランダムに配置することや、実験群後に統制群を5名、統制群後に実験群を5名と順番の人数を統制するというカウンタバランス(順序効果)を制御する必要があります。



2-2 アンケート方法

被験者実験において、エージェント／ロボットを評価する方法として、被験者にエージェント／ロボットに関するアンケートを実施し、アンケート結果に基づいてエージェント／ロボットを評価します。本実験では、基本的なアンケート方法である SD 法と、ロボット専用のアンケート方法である Good Speed Questionnaire、一般的なアンケート手法である五段評価尺度法(リッカート尺度)について説明します。また、それぞれのアンケート方法の様式は、Teams のファルダにアップロードしてあります。

2-2-1 SD 法

SD 法(Semantic Differential 法)はアメリカ心理学者のオズグッド(Osgood, C.)が、概念(対象)の意味を測定するために構築した方法です。評価者の対象に対する感情的な印象やイメージを推定するために使用されます。SD 法は下記のように、七段階もしくは五段階で評価します。また、各項目は基本的に形容詞となっています。評価者が対象に対する感情的な印象やイメージを想定できるように、形容詞の使用が推奨されています。そして、項目は形容詞の対義語で構成されます。例えば、下記の図のように、「非日常的なー日常的な」、「小さいー大きい」、「優しいー怖い」などと形容詞の対義語で構成されます。

SD 法による評価方法は、項目一つ一つを点数化します。例えば、ポジティブな意味を持つ形容詞側の「非常に」を 7 点とし、ネガティブな意味を持つ形容詞側の「非常に」を 1 点として、点数化します。また、各項目に対して因子分析を行い、結果を分析する方法もあります。

	非常に	かなり	やや	いすれで もない	やや	かなり	非常に	
非日常的な	1	2	3	4	5	6	7	日常的な
小さい	1	2	3	4	5	6	7	大きい
優しい	1	2	3	4	5	6	7	怖い

2-2-2 God Speed Questionnaire

God Speed Questionnaire は、ロボットとインタラクションを行った評価者が、ロボットから受けた印象やイメージを推定するために使用します。ロボット専用に構築されたアンケート方法です。下記のような下位尺度、項目数などで構築されています。

下位尺度	項目数	代表的項目
擬人化 (anthropomorphism)	5	機械的 (machinelike) - 人間的 (humanlike) 人工的 (artificial) - 生物的 (lifelike)
生命性 (animacy)	6	活気のない (stagnant) - 生き生きとした (lively) 無関心な (apathetic) - 反応のある (responsive)
好ましさ (likability)	5	親しみにくい (unfriendly) - 親しみやすい (friendly) 不愉快な (unpleasant) - 愉快な (pleasant)
知性の知覚 (perceived intelligence)	5	無能な (incompetent) - 有能な (competent) 無知な (ignorant) - 物知りな (knowledgeable)
安全性の知覚 (perceived safety)	3	不安な (anxious) - 落ち着いた (relaxed) 冷静な (calm) - 動揺している (agitated)

(各項目、相反する形容詞を両極として5段階で評価)

God Speed Questionnaire は、SD 法とは異なり、各項目で印象を評価するのでは、下位尺度における各項目による平均値を、下位尺度の項目の評価値として使用します。例えば、下記のように「擬人化」の下位尺度において、五つの項目があり、それぞれが五段階で評価されます。そして、各項目の点数による平均値が「擬人化」の評価値となります。

擬人化

GODSPEED I: ANTHROPOMORPHISM

Please rate your impression of the robot on these scales:

以下のスケールに基づいてこのロボットの印象を評価してください。

Fake 偽物のような	1	2	3	4	5	Natural 自然な
Machinelike 機械的	1	2	3	4	5	Humanlike 人間的
Unconscious 意識を持たない	1	2	3	4	5	Conscious 意識を持っている
Artificial 人工的	1	2	3	4	5	Lifelike 生物的
Moving rigidly ぎこちない動き	1	2	3	4	5	Moving elegantly 洗練された動き

平均値: 評価者がロボットに対して
感じた擬人化の数値

2-2-3 五段階評価尺度法(リッカート尺度)

5段階評価によるアンケートは、「はい」か「いいえ」では回答しにくい質問に活用されます。これをリッカート尺度と呼び、さまざまなアンケート調査に使用されます。例えば、特定の質問に対し、「強く同意する・やや同意する・どちらともいえない・やや同意しない・まったく同意しない」という選択肢を設定し、意思の度合を評価します。五段階評価尺度法では、自身で項目を決める必要があります。

【質問】：このコンテンツでまた遊びたい？

- 1.とてもそう思う**
- 2.ややそう思う**
- 3.どちらとも言えない**
- 4.あまりそう思わない**
- 5.まったくそう思わない**

● メリット

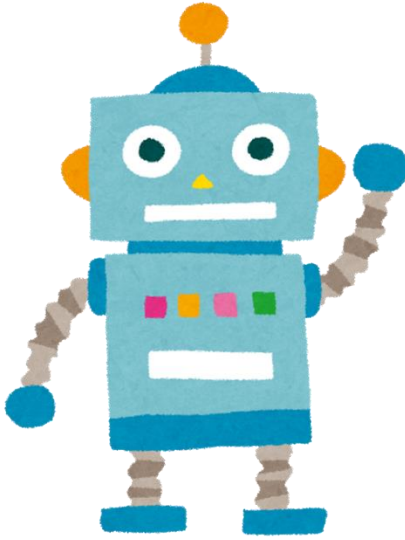
アンケートで5段階評価を活用する利点として、二者択一方式に比べ詳細なデータが収集できる点が挙げられます。回答を5つの段階に分類するため、顧客心理を細かい部分まで把握することが可能です。また、相反する質問項目を抜粋して比較することができる点も利点となります。例えば、「とてもそう思う」に対し、「まったくそう思わない」の割合がどの程度かを集計して分析できます。加えて、「はい・いいえ」では表現しにくい質問に答えやすい点も利点となります。「どちらとも言えない」などの中立的尺度を導入することで、評価者は解答しやすくなります。

● デメリット

5段階評価を用いたアンケートの欠点は、回答が当たり障りのない内容に集中する恐れがある点です。5段階評価のアンケートは、多くの場合「どちらとも言えない」などの中立的な選択肢が用いられます。答えに悩んだ結果、評価者が中立的な選択肢を選ぶケースは多いです。また、回答の選択に時間がかかるため、評価者が途中でアンケートを放棄し、雑に回答することもあります。その結果、求める回答量に達しない・回答の信憑性に欠けるなど、満足のいく調査結果が得られない可能性があります。

2-3 アンケートの作成・選定

コンテンツに対してアンケートを実施する場合は、「目的」を明確にしておく必要があります。つまり、評価して欲しい内容を正確に決めることです。例えば、お笑いのネタを配信するエージェント・コンテンツを開発し、エージェントのネタが面白いかどうかを表したいのであれば、「ネタの面白さ」という項目による五段階評価尺度で評価するといいでしょう。もしくは、エージェント・コンテンツからユーザが感じた全体的な印象を評価したいのであれば、「楽しいーつまらない」、「明るいー暗い」などの形容詞対による SD 法で評価するといいでしょう。このように、目的に応じて使用するアンケート手法や項目を作成／選定する必要があります。また、アンケート手法は複数使用することも可能です。



五段階評価尺度法

【質問】：コンテンツは面白い？

- 1.とてもそう思う
- 2.ややそう思う
- 3.どちらとも言えない
- 4.あまりそう思わない
- 5.まったくそう思わない

SD法

非常に 非日常的な 小さい 優しい	かなり	やや	いすれ もないで	やや	かなり	非常に 日常的な 大きい 怖い
1	2	3	4	5	6	7

何を評価して欲しい??

➤ 作成／選定の手順

(ア) 何を評価したいか目的を決める(複数でも良い)

(イ) それに適したアンケート手法を選定

(ウ) アンケート手法内の項目を作成

(エ) 項目数やアンケートの数を確認し、不要なものを削除

基本手には目的の数に応じて(ア)～(ウ)を繰り返し、(エ)で調整します。

第2週：課題

エージェント・コンテンツを被験者実験で評価しよう！！

目的

第1週のエージェント・コンテンツをアンケート手法により評価

評価基準

- アンケートの選定理由を記載できているか
- アンケートより得られた数値をわかりやすくグラフ化できているか
- アンケート結果からエージェント・コンテンツを評価できているか

実験手順

1. アンケート手法の作成・選定(→3 限)
 2. 被験者1名以上にエージェント・コンテンツとのインタラクション(→4 限以降)
 3. 被験者にアンケートの解答を指示
 4. エクセルにてアンケートを集計し、グラフ化
 5. アンケート結果を基に、エージェント・コンテンツを評価
- 基準点を中間点とする。五段評価尺度法では1~5点となるので、中間点は2.5点となる。この中間点とアンケートから「平均点」も算出し、比較する。

提出方法・べ切

- ✧ レポート(PDF)をメールにて提出
- ✧ PDF名は「学籍番号」にすること
- ✧ 提出先：jimenez@cis.aichi-pu.ac.jp
- ✧ 件名：コンテンツ・第2週レポート
- ✧ 提出べ切：第3週目の実験日前日 16:00

注意点：

- ✓ 友人と話し合いながらアンケートを選定しても良いが、一人一人でアンケートを一つ以上は作成してください。
- ✓ 被験者となる場合は、適当にアンケートを記載するのではなく、自身の主観でしっかりと評価してください。

第3週：検定／分析方法

3-1 検定

アンケートで得られた評価値は、評価者が付けたものであるため主観的な要因が強くなります。そのため、評価値が高いかどうかを客観的に評価するために、統計的な検定を評価値に対して実施する必要があります。

3-3-1 検定とは

- 二つ以上のデータ数値により立てられた仮説が発生する確率を求め、その仮説が正しいかどうかを確認する方法
- 有意水準を基に、帰無仮説もしくは対立仮説が発生する確率を求め、どちらの仮説が正しいかどうかを決める方法

有意水準(5%, $\alpha = 0.05$)

- 帰無仮説を破棄するかどうかを決めるための水準
 - 帰無仮説が発生する確率が水準よりも低い場合は、仮説は正しくない
- 有意水準は、研究分野に応じて異なります。例えば、医療分野では、1%で扱われることが多いです。また、ビジネスでは10%まで考慮する場合があります。10%まで考慮する場合は、「有意傾向」として評価します。

帰無仮説($x_1 = x_2$)

- データ A の計算値 x_1 とデータ B の計算値 x_2 が等しいという仮説

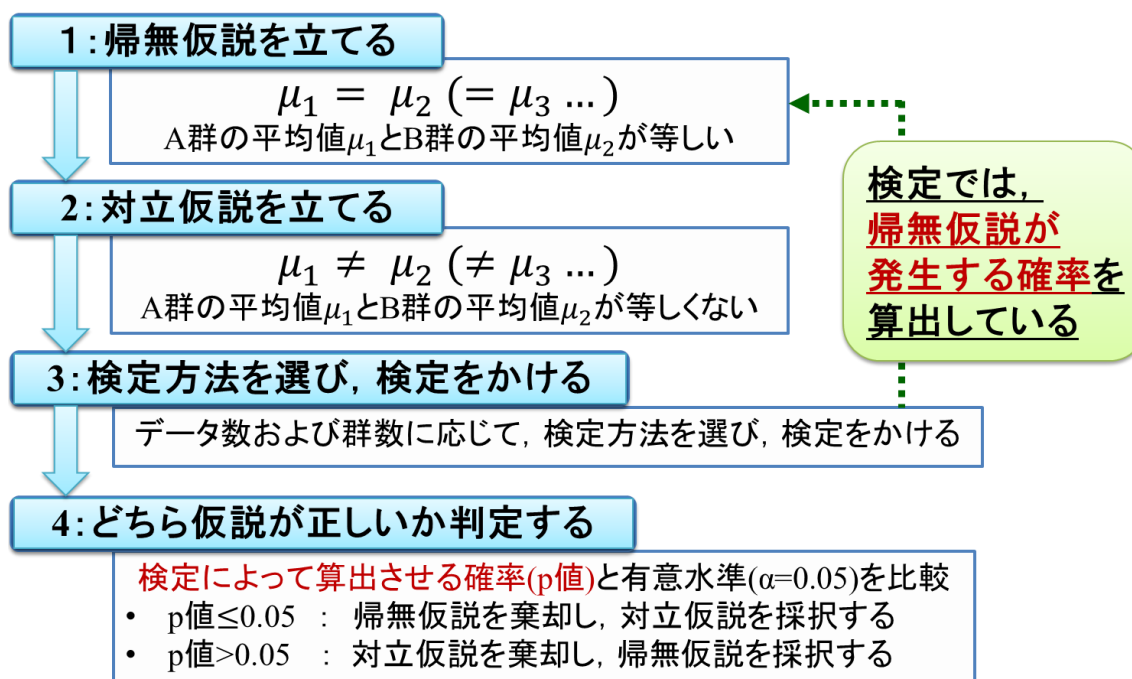
対立仮説($x_1 \neq x_2$)

- データ A の計算値 x_1 とデータ B の計算値 x_2 が等しくないという仮説

p 値

- 検定を実施することで、算出される帰無仮説の発生確率
- 評価値：3.6, 基準値：2.5, p 値：0.03 (<0.05)
→有意差あり→統計的に評価値は、基準値に比べて高い
- 評価値：2.0, 基準値：2.5, p 値：0.03 (<0.05)
→有意差あり→統計的に基準値は、評価値に比べて高い
- 評価値：3.0, 基準値：2.5, p 値：0.07 (>0.05)
→有意差なし→統計的に評価値と基準値は同じ

3-3-2 検定の流れ



3-3-3 母分散が未知の場合の母平均の検定

検定には様々な種類がありますが、本実験では一つの群を対象に実施できる「母分散が未知の場合の母平均の検定」について説明します。本検定は、エクセルファイルにて実験を実施できます。この検定は、下記のような状況で使用されます。

ある工場のある製品のラインを改変し、製品9個を標本として取り出し、あるパラメータを計測したところ以下の値が得られたとする。

3.3, 3.2, 3.4, 3.3, 3.5, 3.2, 3.5, 3.2, 3.5

平均値 $\bar{x}=3.34$ であった。この製品ラインの基準値 $\mu_0=3.25$ であった。改変後のパラメータは基準値を超えているか。

検定を行うには、帰無仮説と対立仮説を立てる必要があります。前述した状況の下記のような仮説が立てられます。そして、平均値やデータ数に応じて検定統計量を算出します。今回の検定では、検定統計量ではu値を算出します。u値をp値に変換することで、帰無仮説が発生する確率を算出できます。検定統計量におけるVeは不偏分散、nはデータ数を示しています。

仮説：帰無仮説 $\bar{x} = \mu_0$, 帰無仮説 $\bar{x} \neq \mu_0$

$$\text{検定統計量} : u = \frac{\bar{x} - \mu_0}{\frac{s_e}{\sqrt{n}}}$$

エクセルで検定を行う場合は、下記のようにエクセルシートに必要な数値を記載し、算出していきます。平均は average()関数、不偏分散は var.s()関数(算出されるものは二乗値)、平方根は sqrt()関数、絶対値は abs()関数を使用することで算出できます。n はデータ数を示しています。

	A	B	C	D
1	母平均の検定(母分散未知)			
2				
3	入力	有意水準(両側) α	μ_0	
4		0.05	3.25	
5	計算	n		
6		9		
7		計測値 xi		
8		3.3		
9		3.2		
10		3.4		
11		3.3		
12		3.5		
13		3.2		
14		3.5		
15		3.2		
16		3.5		
17		平均	不偏分散 s_e^2	検定統計量 t
18		3.344	0.0178	2.125
19	出力	p値	=T.DIST.2T(D18,B6-1)	
20		0.0663		

p 値=0.06 のため、有意水準 0.05 以上です。そのため、対立仮説が棄却され、帰無仮説が採択されます。結果として、改変後のパラメータは基準値を超えていないと言えます。

第3週：課題

アンケート結果に検定を実施して分析しよう！！

目的

第2週のアンケート結果に対して検定を実施して，結果を再度分析

評価基準

- 検定内容を理解し，検定結果に基づいてコンテンツを評価できているか
- 第2週のレポートつまり検定をかけない場合と比較できているか
- 検定結果を基にコンテンツの改良方法や発展先を記載できているか

実験手順

6. アンケートで得られたデータを基に検定を実施
7. 検定結果を基に，エージェント・コンテンツを評価
 - 有意水準：0.01, 0.05 とする．つまり， p 値：0.004(<0.01)だった場合は，統計的に評価値と基準値間是非常に差があるといえる． p 値：0.03(<0.05)だった場合は，統計的に評価値と基準値間には差があるといえる．
 - 有意傾向：0.1 とする．つまり， p 値：0.07(<0.1)だった場合は，統計的に評価値と基準値間には現状として差がないが，差がでる傾向であるといえる．
8. エージェント・コンテンツの利点・欠点を考察
9. 利点を活かした発展先や欠点の改良方法を考察

提出方法・べ切

- ✧ レポート(PDF)をメールにて提出
- ✧ PDF 名は「学籍番号」にすること
- ✧ 提出先：jimenez@cis.aichi-pu.ac.jp
- ✧ 件名：コンテンツ・第2週レポート
- ✧ 提出べ切：次実験の実験日前日 16:00

注意点：

- ✓ 友人と話し合いながら検定をかけてもよいが，エージェント・コンテンツが一人一人異なるので，考察は自身で考えてください．

参考文献

- [1]. GUI とは？初心者向けに分かりやすく 1 分で解説, <https://it-biz.online/it-skills/gui-cui/#toc2>
- [2]. 日向 俊二, Java GUI プログラミング Swing を使った今どきのアプリ開発, 株式会社カ
ットシステム, 2020.
- [3]. テキスト読み上げ テキストトーク, <https://gui.jp.net/textalk/>
- [4]. 石黒浩, 宮下敬宏, 神田崇行, コミュケーションロボット 人と関わるロボットを開発
するための技術, オーム社, 2005.
- [5]. Bartneck, C., Croft, E., Kulic, D. and Zoghbi, S.: Measurement instruments for the
anthropomorphism, Animacy, likeability, perceived intelligence, and perceived safety of
robots, Int. J. Social Robotics, Vol.1, No.1, pp.71-81, 2009.
- [6]. 金田行雄, 笹井理生, 古橋武, 統計・多変量解析とソフトコンピューティング—超多自
由度系解析を目指して— 改訂版, 共立出版, 2014.

A. 音声処理

Matlab などを用いた計算機演習を通じ、音の合成、音声分析と合成を体験的に学ぶ。同時にMatlab の操作方法を習得する。

1 Matlab の紹介

Matlab は、科学技術計算のための高性能な言語である。数値計算、グラフィックス、プログラミング等が1つの環境に統合されており、簡単なコマンドを用いて、一般的なデータ処理、演算から結果の可視化まで行うことが出来る。Matlab の入門には、別配付の資料、MATLAB オフィシャルマニュアルの他、文献[1] [2] を参照すること。また、オンラインマニュアルを活用すること。英文が苦手な者は辞書を持参のこと。

以下の説明において、

```
>>
```

は、Matlab の環境を表す。また、

```
%
```

は、シェルの環境を表す。

Matlab 環境へは、コマンドを直接打ち込めるけれども、ごく簡単なコマンドを除いて、そうしないほうがよい。コマンドはエディタで編集した後、Matlab 環境へカットアンドペーストしよう。またはエディタで編集したスクリプトは .m の拡張子をつけてファイルに保存し、このファイル名で実行しよう。複数行のコマンドを一度に処理出来る上、エラーがあってもコマンドがエディタの画面やファイルに残っているので、対処しやすい。また、入出力は関数の引数として設定することもできる。将来、Matlab スクリプトや関数を自分で書くときの参考にもなる。なお、Matlabでは同じ環境で変数が新しい値に代入されるまでは、基本的にその値が継承されることに注意しよう。

2 音の合成と分析

2.1 純音の合成と聴取

最も単純な音は正弦波である。倍音を含まないため、純音ともよばれる。基本周波数440Hz の純音を作り、試聴してみよう。

Matlab 環境で音を正しく再生させるためには、音データのサンプリング周波数を8000Hz に設定する必要がある。また、最大振幅は 1.0 である。ここではマージンを見て、振幅を0.9 としよう。再生用の関数はsound である。さらに波形をプロットし、正弦波であることを確かめる。

```
>> % How to synthesize sin wave
>> Fs = 8000; t=1/Fs:1/Fs:1; f0=440;
>> x = 0.9*sin(2*pi*f0*t);
>> sound(x);
>> plot(t,x), axis([0 0.01 -1 1]);
```

練習課題1.1: 基本周波数を変えて、ピッチが変わることをたしかめよ。サンプリング定理を参考しながら、基本周波数をナイキスト周波数(サンプリング周波数の半分) より大きくするとどうなるか? 振幅を変えて、音の大きさが変わることをたしかめよ。振幅を1.0 より大きくするとどうなるか?

2.2 エンベロープ

前節の音は、音の開始と終了の際にプチッという音が生じ、耳障りである。これは、音が急に発生し、消滅するためである。エンベロープを付けて、音の立ち上りをなめらかにしよう。適当な関数、例えばcos を使って、なめらかなエンベロープを作り、元の信号と積を取る。下の例でenvの中身をプロットして確認しよう。

```
>> % envelope
>> tRise=0.3; nRise=Fs*tRise;
>> env=ones([1 length(t)]);
>> env(1:nRise)=0.5*(1-cos(pi*t(1:nRise)/tRise));
>> xEnv=env.*x;
>> sound(xEnv);
```

練習課題1.2: 音の立ち下がりにもエンベロープをかけ、なめらかに減衰するようにせよ。立ち上りと立ち下がりの時間を色々変えて、音を聴取するとともに、波形をプロットせよ。

2.3 色々な音の音色とそのスペクトル

Matlab には色々な信号を発生する関数が用意されている。

作成される音のスペクトルを表示すると、基本周波数の成分以外に、整数倍の周波数を持つ成分があることが分かる。これらは倍音と呼ばれる。倍音が含まれる音は複合音と呼ばれる。正弦波を重ね合わせるにより、いろいろな音色の音を作ることが出来る。6次倍音までを使って、音を合成する。n 次倍音の振幅 amp(n), 倍音周波数 har(n)*f0, 位相 phase(n) を適当に与えて、合成音を作る。

演習課題1.3: 複合音において、倍音の振幅、位相が変われば音にどんな変化が現れるか? 波形やスペクトルにはどんな変化が現れるか? 倍音周波数が基本周波数の非整数倍になれば、どんな変化が現れるか?

```

>> % sinusoidal model
>> amp=[1 0.8 1.5 0.2 0.9 0.4];
>> har=[1 2 3 4 5 6];
>> phase=[0 pi 0.1*pi 0.5*pi -0.2*pi 1.3*pi];
>> x=zeros([1 length(t)]);
>> for n=1:6
        xComp(n,:)=amp(n)*sin(2*pi*har(n)*f0*t+phase(n));
        x=x+xComp(n,:);
    end
>> x=0.9*x/max(x).*env;
>> plot(t,x), axis([0.5 0.51 -1 1]), sound(x);
>> f=(0:length(t)-1)*Fs/length(t);
>> plot(f,abs(fft(x)));

```

2.4 スペクトログラム

時間的に変化する音の場合、短い時間間隔でスペクトル分析を行い、刻々変わるスペクトルを時間的に追いつながら分析を行う必要がある。分析結果の表示には、時間を横軸、周波数を縦軸に取り、強度を色で表すなど工夫が必要である。この表示されるグラフをスペクトログラム(ソナグラム)と言う¹。Matlab にはスペクトログラムを描く関数 `specgram` が用意されている(時間・周波数分析)。

周波数が連続的に変化する正弦波を作る関数 `chirp` を使い、分析用サンプル信号を作る。そのスペクトログラムを描く。

```

>> y=0.9*chirp(t,0,1,440)+0.3*chirp(t,0,1,880); % 0Hzからt=1秒で 440Hz に到達
>> y=0.9*y/max(y), sound(y);
>> specgram(y,256,Fs);axis([0 0.95 0 1000]);

```

`chirp` 関数の引数は順に、時間変数、時刻 0 における周波数、到達する時刻、その時刻の周波数を表す。短時間フーリエ変換 (STFT) を使ったスペクトログラム `specgram`(現在は `spectrogram` が使用可) 関数の引数は順に、分析信号、窓分析時間(データ数)、サンプリング周波数である。この分析時間(データ数)はサンプリング周波数によって変化する必要がある。

2.5 音データのファイル入出力

Matlabで作った音をファイルに書き出そう。Unixで古くから用いられて来た音データの標準形式は `au` フォーマットと呼ばれる形式である。現在いろいろなフォーマットの音声データが利

¹ 刑事もののドラマで出て来る声紋診断装置はこのスペクトログラムを出力する装置にはかならない。

用されている．その中でwavフォーマットは典型的なもののひとつである．

```
>> [xGet, Fs] = audioread('my1stSound.wav');
```

以上はmy1stSound.wavというファイル名からwavフォーマットの音声データを読み込む，音声データxGet，サンプリング周波数Fsを得る．

また，以下は音声データY，サンプリング周波数Fsの情報をwavフォーマットの音声ファイル名 mysynthesis.wavに書き出す操作である．

```
audiowrite('mysynthesis.wav', Y, Fs);
```

音声全般については，参考文献「音声工学」[3]を参照することとする．またいろいろな音声処理フリーソフトがある．waveSurfer[4]とparrrt[5]はその中の2つである．Fig.1に示す音声分析フリーソフトwaveSurfer[4]を利用したファイルの入出力や分析も参照してみよ．図中音声波形，スペクトログラム，フォルマント，ピッチ，パワーが描かれている．

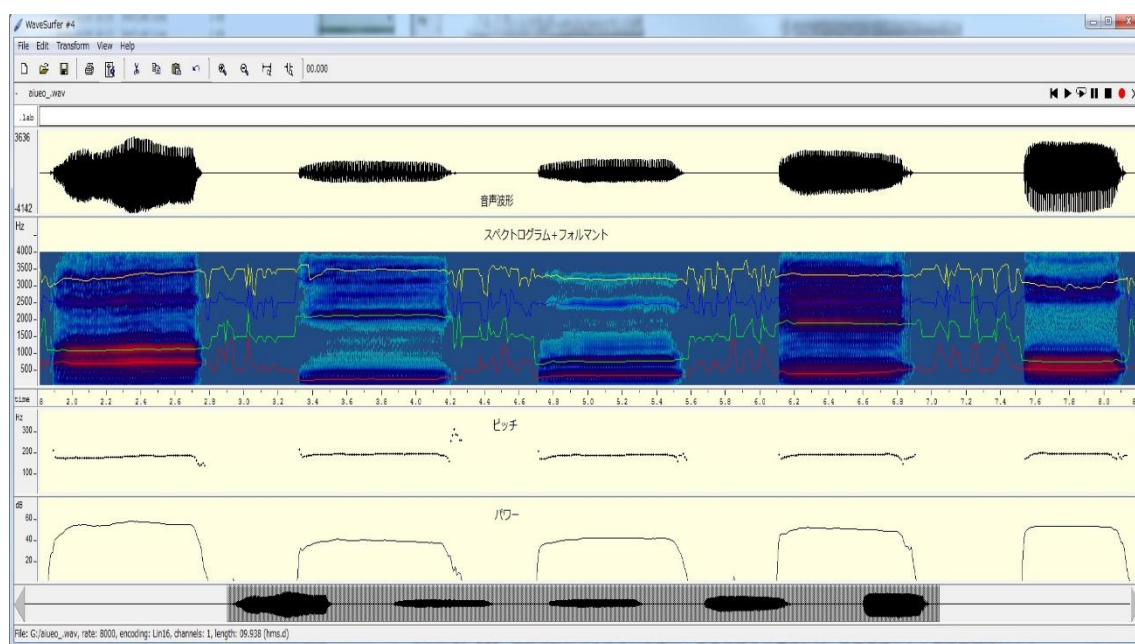


Fig.1 音声分析フリーソフトwaveSurfer[4]の使用例(発声/a, i, u, e, o/の場合)

レポート課題(第1週)

1. 演習課題1.1-1.3について報告しよう．
2. 出来るだけ変わった音を作り（または入手し），その時間波形，スペクトル，スペクトログラムを表示せよ．分析結果と音色の関連について考察せよ．

3 声の分析

3.1 マイクからの録音

ワークステーションに接続するマイクから音を録音し、ファイルに格納する方法を解説する。以下は8ビット量子化、8kHzサンプリング、モノラルで録音する方法である。

```
% rec -b 8 -r 8000 -c 1 my1stSound.wav
```

wav形式の音ファイルは下記コマンドで再生できる。

```
% play my1stSound.wav
```

演習課題2.1: 言葉や歌などを自由に発声し、録音せよ。再生してみよ。Matlab を用いて波形を観測せよ。リファレンスとして/a, i, u, e, o/ の5母音を少し間隔を持って1つのファイルに録音し、波形を表示せよ。/r, l, th, p, s/ 等の子音も含めて録音し、波形を観測せよ。自由発声の中で母音や子音が発声されている部分の波形を特定し、リファレンスの波形と比較せよ。波形の観察からどれくらい似ているかを考察しよう

3.2 母音のスペクトル分析

前節の課題で録音した5母音のスペクトル分析を行う。個々の母音を切り出そう。Fsが8000Hzの母音 /a/ が 1 秒間 8000 点のデータとして切り出せたとする。分析例をしめす。

```
>> Nd=8000; sigA=sig(1:Nd); sigAF=fft(sigA(4000:5999));  
>> Fs=8000; t=1/Fs:1/Fs:Nd/Fs; f=(0:length(t)-1)*Fs/length(t);  
>> subplot(2,2,1), plot(t,sigA),title('Analysis of vowel /a/');  
>> xlabel('Time (sec)'),ylabel('Raw signal');  
>> subplot(2,2,3), plot(f,20*log10(abs(sigAF))),axis([0 Fs/2 -70 50]);  
>> xlabel('Freq (Hz)'), ylabel('Mag (dB relative)');  
>> subplot(2,2,[2 4]), specgram(sigA,256,Fs),axis([0 1 0 4000]);  
>> title('Spectrogram of vowel /a/');  
>> xlabel('Time'), ylabel('Frequency');
```

波形、周波数スペクトルおよびスペクトログラムを描いた。スペクトル分析では通常、スペクトルの強度をデシベル²で表示する。

² dB(デシベル)=B(ベル)/10. ベルとは、音の大きさを表す相対的な単位である。ある基準のエネルギーより 10 倍大きなエネルギーをもつ音はその基準に比べて 1 ベル高い強度を持つと定義される。信号の

Fig.2に示す分析結果を考察しよう。スペクトルには等間隔に並んだピークが現れている。母音/a/は倍音構造を持っていることを意味する。これは、波形が周期的であることに対応している。最も左のピークが基音、それ以外のピークが倍音である。基音の周波数は約180 Hz付近である

スペクトルピークをなめらかに結んだエンベロープを描こう。750 Hz, 1100 Hz, および2500 Hzあたりにピークがある。これらはフォルマントと呼ばれる。ただし、ここでFig.2のスペクトルは約0.25秒間の音声に対するFFTで計算したものである、この間の平均スペクトルになる。もし、ぼやけている或いははっきりしていないとき、窓長の調整などを行う必要がある。

周波数の低いピークから順に第1, 第2, ... フォルマントと呼ぶ。F1, F2, ... で記述する。スペクトログラムでは、倍音は横縞で表されている。縞の濃い部分がフォルマントに対応する。

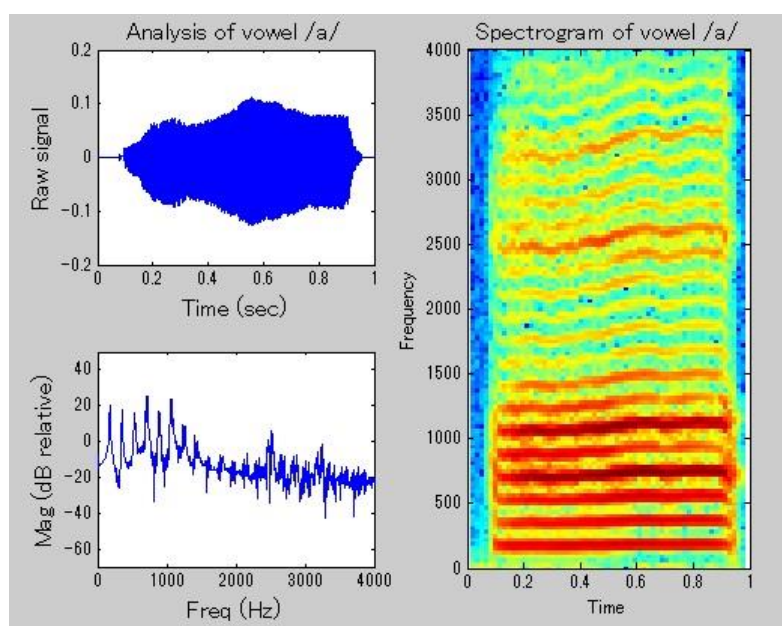


Fig.2 母音/a/の音声波形、スペクトルとスペクトログラム

演習課題2.2: 自分の5母音のスペクトルまたはスペクトログラムをもとに、分析せよ。

- ・各母音の基本周波数を求めよ。
- ・フォルマント構造は母音の種類によりどのように異なるかを確認、フォルマント周波数 (F1, F2, ...) を求めよ。

3.3 音声の科学

前節の分析結果をもうすこし詳しく理解するためには、母音の発声原理を知らなければならない。発声器官は大きく分けて、声帯と声道の2種類から構成される。母音の発声では、必ず声帯が振動し、これが音源となる。この振動は周期振動であり、声帯音源は倍音構造をもつ。声帯のまわりの筋肉をコントロールすることで、振動周波数(基本周波数)を変えることができる。男性は声帯が長いのでピッチの低い声、女性は短いので高い声を発する。声帯音源は倍音が多く含まれたブザーのような音である。一方、声道は音源の周波数特性を変えるフィルタの役目を果たす。(下の

振幅が2倍になれば、エネルギーは4倍になるので、振幅の $\log$ を取り、それを20倍すればデシベル値が得られる。上記の例では、振幅が1の場合が基準の強度(= 0 dB)である。

Fig.3イメージ図参照.) このフィルタはフォルマントを作る. すなわち, 特定の倍音を強める働きをする. フィルタの特性は声道の形状を変えることにより変化する. 仮に, 声道が長さ 17 cm の円筒としよう. 音速は340 m である. 声道は一端が閉じられた管と考えられるから, $\lambda/4, 3\lambda/4, \dots$ の共鳴がある. 共鳴周波数は $340/(0.17 \times 4) = 500 \text{ Hz}$, $3 \times 340/(0.17 \times 4) = 1500 \text{ Hz}$ と推定される. これらは, 第1, 第2フォルマント周波数となる. 実際の声道は円筒ではないから, その形状によって, フォルマント周波数は変化する. 声道形状とフォルマント周波数の関係は, 音響学の知識なしには, 理解が難しい. しかし, あごの動き, 舌を使って, 声道形状を変え, 母音を発声し分けしていることは日常経験することである.

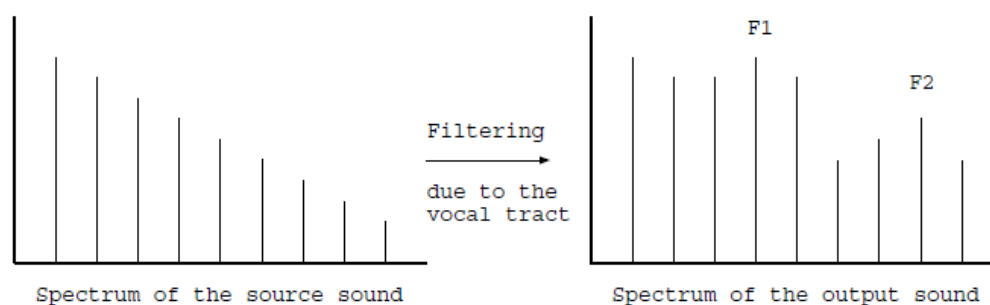


Fig.3 声道は音源の周波数特性のイメージ図

実は, 第1, 第2フォルマント周波数によって母音が識別されることが分かっている. 第1, 第2フォルマント周波数を変化させた音を聞かせ, どの母音に聞こえるかの心理実験³を日本人に対して行くと, 以下のFig.4(左)のようになる. 即ち, 典型的な母音 /a/ は, $F_1 = 800 \text{ Hz}$, $F_2 = 1200 \text{ Hz}$ 程度, 典型的な母音 /i/ は, $F_1 = 300 \text{ Hz}$, $F_2 = 2500 \text{ Hz}$ 程度である. 同図(右)は分析結果である.

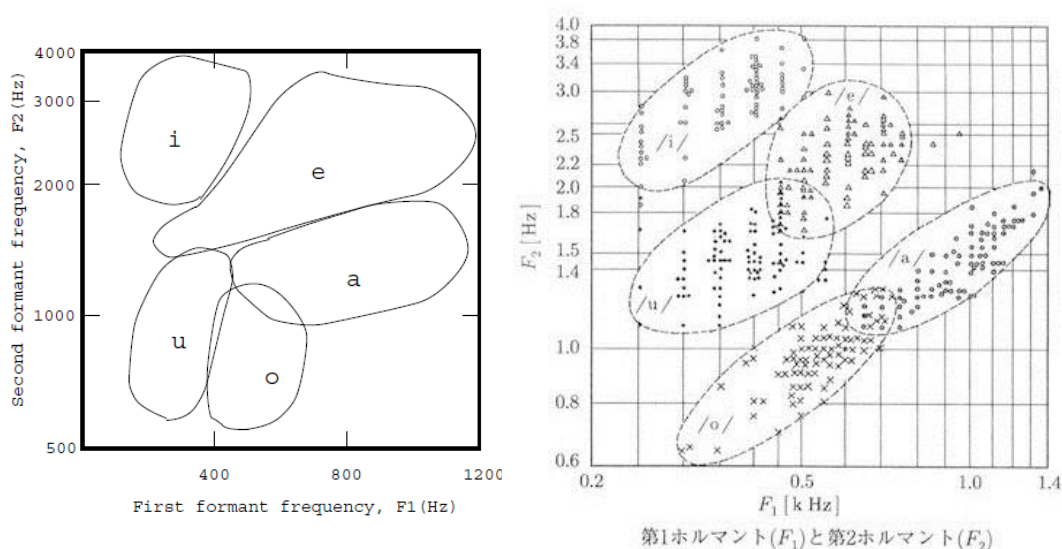


Fig.4 日本人の単母音のF1, F2 分布図[3]

³ 興味深いことに, 生まれてからの言語環境によって, 母音の‘地図’が出来上がる. 新生児は, どの言語に対しても全く白紙の状態である. ところが, 英語環境にさらされると, 8~10 個, スウェーデン語だと 12 個の母音の区分けが起こる. この言語への適応は生後 6 ヶ月の乳児に対しても見られる.

3.4 母音のLPC 分析

LPC(linear prediction coding: 線形予測符号化) とは, 信号のある時刻における値を, それ以前の複数の時刻における値の線形和で予測するモデル化のことである. すなわち, ある信号 x_n があったとき, その時系列から, その時系列自身を出力するような, IIR フィルタを予測することである. $N + 1$ 次のフィルタを考え, その係数を $a_1 = 1, a_2, \dots, a_{N+1}$ とすると,

$$x_k = -a_2x_{k-1} - a_3x_{k-2} - \dots - a_{N+1}x_{k-N-1}$$

となる, LPC 分析とは, 観測信号 x_n からフィルタ係数 a_n を予測することである.

音声信号に対してLPC 分析を行うと, 声道フィルタの特性⁴, すなわちフォルマント特性が推定出来る. さらに, そのフィルタに, 音源波形を模したパルス列を入力すると, 音声合成できる. この節ではフォルマント推定を行う. 音声合成は次週の演習である. Matlab には, LPC 分析を行う関数 `lpc` が組み込まれている. 母音 /a/ をLPC 分析しよう.

```
>> Fs = 8000;
>> tapN = 12; NanaI = 0.6*Nd+1; NanaF = 0.8*Nd;
>> Ndd=length(sigA(NanaI:NanaF));
>> t = 1/Fs:1/Fs:Ndd/Fs; f=(0:length(t)-1)*Fs/length(t);
>> sigAF = fft(sigA(NanaI: NanaF));
>> asigA = lpc(sigA(NanaI:NanaF),tapN); hsigA = freqz(1,asigA,f,Fs);
>> plot(f, 20*log10(abs(sigAF)), 'm--'); axis([0 Fs/2 -70 50]);
>> title('Spectrum and formant of vowel /a/');
>> xlabel('freq (Hz)'), ylabel('mag (dB relative)'), hold on
>> plot(f, 20*log10(abs(hsigA))+15); hold off
```

以上の例では, 予測するフィルタのタップ数(次数) `tapN` を12 とする. また, 分析は, 音声信号の振幅が大きい0.6 から0.8 秒について行う. 分析の結果得られたフィルタの係数から, フィルタの特性を計算するために `freqz` 関数が用意されている.

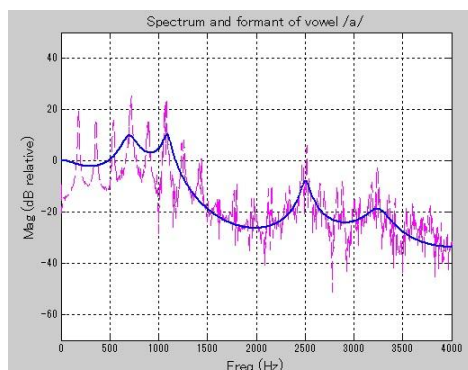


Fig.5 男性話者母音/a/のスペクトルとホルマント抽出

⁴ 厳密に言うと, 得られたフィルタには声道フィルタの特性と音源の周波数特性の両方が含まれる. そのため, LPC 分析されたフィルタを使って音声合成する場合には, 声帯音源として周波数特性のフラットなパルス列を用いる必要がある.

フォルマント形状が実線でプロットされる。参考のため、スペクトル分析結果も同時に表示する。フォルマント周波数を求めるためにグラフから任意の点の x , y 座標を求められれば便利である。そのために, `ginput` 関数が用意されている。使い方は, `ginput(1)` と入力し, リターンキーを押す。マウスポインタを第1フォルマントのピークに置き, 左ボタンを押すと, x , y 座標が返される。 $F_1 = 746.5 \text{ Hz}$ と求まる。

```
>> ginput(1)
ans =
746.5438    23.1579
```

さらに Fig.6に示す音声分析フリーソフトwaveSurfer[4]を利用した音声分析も参照してみよ。

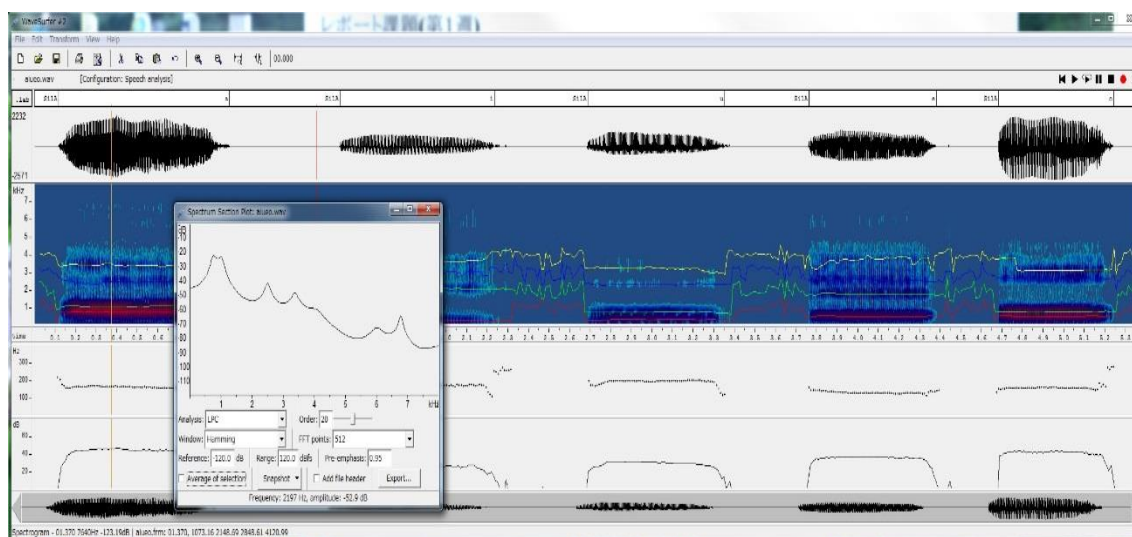


Fig.6 WaveSurfer[4]による男性話者母音/aのLPCスペクトル

演習課題2.3: 自分の発声した母音をLPC 分析し, フォルマント周波数(F_1 , F_2 , F_3) を求めよ。分析のタップ数を色々変えてみよ。タップ数2個で1つのフォルマントピークが推定出来ることが知られている。そのことを確かめよ。

レポート課題(第2週)

1. 演習課題2.1-2.3について報告しよう。
2. 自分の発声した5母音の分析結果と音声の科学の節で紹介した理論とを比較する。理論に基づいて自分の分析結果を説明せよ。もし, 理論と異なる結果が得られたのであれば, その原因を考察せよ。
3. 実験に使用した収録や作成した音声ファイルは添付して報告すること。

4 声の合成

4.1 フィルタの基礎

音声の科学の節で述べたように、声道は声帯音源波の周波数特性を変化させるフィルタの役目を果たす。そこで、まずフィルタ概念を復習しよう。

ある線形システム(声道) に、時系列(声帯音源波) $x_1, x_2, \dots, x_N$ が入力された時、時系列(音声) $y_1, y_2, \dots, y_N$ が出力されたとする。線形システムは、次の式を満たす2組のフィルタ係数 $a_1 (=1), a_2, \dots, a_{n_a+1}$ および $b_1, b_2, \dots, b_{n_b+1}$ で特徴づけられる。

$$y_n + a_2 y_{n-1} + \dots + a_{n_a+1} y_{n-n_a} = b_1 x_n + b_2 x_{n-1} + \dots + b_{n_b+1} x_{n-n_b}$$

線形システム、つまり、フィルタ係数 a_n, b_n が既知であり、入力時系列 x_n がシステムに与えられた場合、出力時系列 y_n を求める方法をアルゴリズム的に書けば、

$$y_1 = b_1 x_1$$

$$y_2 = b_1 x_2 + b_2 x_1 - a_2 y_1$$

$$y_3 = b_1 x_3 + b_2 x_2 + b_3 x_1 - a_2 y_2 - a_3 y_1$$

...

となる。もし $n_b = 0$ のとき、つまり b_1 以外の b_n がゼロのとき、フィルタはIIR(Infinite Impulse Response)、全極型、または AR (AutoRegressive) フィルタと呼ばれる。もし $n_a = 0$ のとき、つまり a_1 以外の a_n がゼロのとき、フィルタはFIR (Finite ImpulseResponse)、全ゼロ型、またはMA (Moving Average) フィルタと呼ばれる。 $n_a, n_b > 0$ なら、ARMA フィルタと呼ばれる。

入力時系列 x_n のスペクトル(周波数特性) を X_k とすれば、出力時系列 y_n のスペクトル(周波数特性) を Y_k は次の式で与えられる。

$$Y_k = H_k X_k$$

ここで、 H_k はフィルタの周波数特性であり、

$$H_k = \frac{b_1 + b_2 z^{-1} + \dots + b_{n_b+1} z^{-n_b}}{a_1 + a_2 z^{-1} + \dots + a_{n_a+1} z^{-n_a}}$$

と表される。ただし、

$$z = e^{2\pi i(k-1)/N}$$

である。上式は、入力信号のスペクトルに H_k を掛算すれば、出力信号のスペクトルが求まることを示している。

簡単なフィルタの実例を Matlab を用いて示す。 $n_a = 1$ のIIR フィルタで以下の周波数特性を持つローパスフィルタを作る。

$$H_k = \frac{1}{1 - 0.9z^{-1}}$$

フィルタの特性を示した後、フラットな周波数特性をもつパルスを入力した時の応答(インパルス応答) を計算する。以下で、freqz はフィルタの特性を計算する関数、filter はフィルタの応答を計

算する関数である. 次に, $n_a = 2$ で, 周波数特性が

$$H_k = \frac{1}{(1 - \alpha z^{-1})(1 - \alpha^* z^{-1})}, \quad \alpha = 0.9e^{2\pi i f_r / f_s}$$

となるIIR フィルタをつくる. f_r, f_s はそれぞれフィルタの共鳴周波数, サンプル周波数である. このフィルタの周波数特性は f_r に1つのピークを持つ.

```
>> %lowpass filter
>> Fs=8000; t=1/Fs:1/Fs:1; f=(0:length(t)-1)*Fs/length(t);
>> a=[1 -0.9]; b=1; h=freqz(b,a,f,Fs);
>> plot(f, abs(h)),xlabel('freq (Hz)'), ylabel('mag (linear)')
>> title('Low pass filter');
>>
>> x=zeros(1,length(t)); x(1)=1;
>> xF=fft(x); plot(f,abs(xF))
>> xlabel('freq (Hz)'), ylabel('mag (linear)')
>>
>> y=filter(b,a,x); yF=fft(y); plot(f,abs(yF))
>> xlabel('freq (Hz)'), ylabel('mag (linear)')
>>
>> % 2nd order filter
>> fr=1000; alpha= 0.9*exp(2*pi*i*fr/Fs);
>> a=[1 -(alpha+conj(alpha)) abs(alpha)^2]; b=1;
>> h=freqz(b,a,f,Fs);
>> plot(f,abs(h))
>> plot(f, abs(h)),xlabel('freq (Hz)'), ylabel('mag (linear)')
>> title('2nd order filter');
```

演習課題3.1: 上記例で $a=[1 \ 0.9]$ とすればハイパスフィルタが作れることを確かめよ. 0.9 を変化させれば何が起こるか? f_r を色々変えて2次形のフィルタの特性を確かめよ. ピークの鋭さはどうすれば変えられるか?

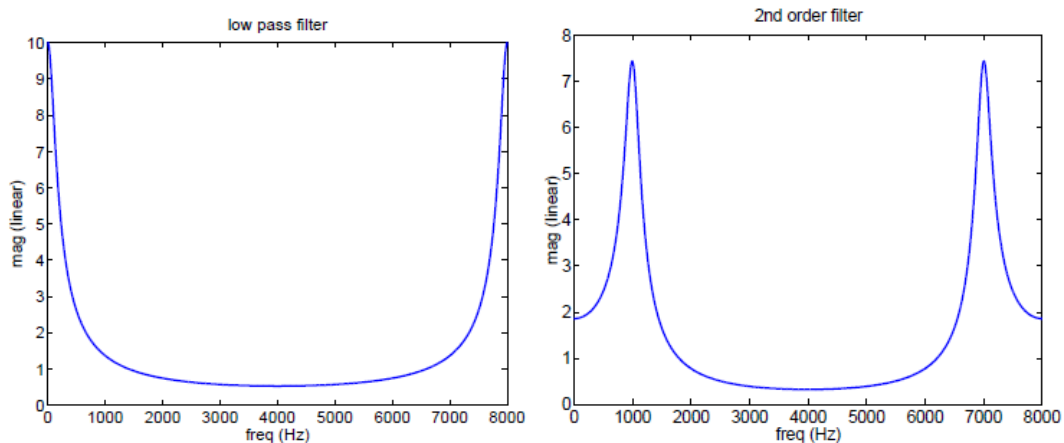


Fig.7 フィルタの周波数特性の例

4.2 音源の設計

音声合成にとって、声道フィルタとともに重要なものは声帯音源である。声帯音源波はパルス列でモデル化出来る。パルスの周期を音声の基本周期と一致させる。パルス列の周波数特性はフラットである⁵。

アナログ信号の場合、パルスはデルタ関数 $\delta(t)$ で表される。デジタル信号の場合には、サンプリング周波数 f_s が有限であるため、パルスは有限の広がりを持ち $\text{sinc}(tf_s)$ で表される。ただし $\text{sinc}(x)$ は $\text{sinc}(x) = \sin(\pi x)/(\pi x)$ で定義される、 sinc 関数である。

基本周波数 $f_0=122$ Hz のパルス列を作る。 sinc 関数は Matlab においても sinc という名前の関数として用意されている。出来たパルス列にエンベロープを付け、試聴する。

```
>> Fs=8000; f0=122; t=1/Fs:1/Fs:1;
>> f=(0:length(t)-1)*Fs/length(t);
>> x=zeros(1,length(t)); tp=1/Fs;
>> while tp < 1, x=x+sinc((t-tp)*Fs); tp=tp+1/f0; end
>> tRise=0.3; nRise=Fs*tRise; env=ones([1 length(t)]);
>> env(1:nRise)=0.5*(1-cos(pi*t(1:nRise)/tRise)); xEnv=env.*x;
>>
>> plot(t,xEnv); sound(0.9*xEnv)
>> plot(f,abs(fft(x)))
```

演習課題3.2: 基本周波数を色々変えて声帯音源音を作り、試聴せよ。

⁵ 実際の声帯音源は高次倍音の強度がなだらかに減少する周波数特性を持つ。しかし、今回の演習のようなソースフィルタモデルを用いた音声合成では、フィルタには、声道の伝達特性の他に、声帯音源の特性を含ませて考える。そのため、周波数特性のフラットな音源を用いる。

4.3 母音の合成I

声道フィルタは, LPC 分析により推定したIIR フィルタが使える. 声の分析の章, 特にLPC 分析の節を参照のこと.

```
>> Fs = 8000; t = 1/Fs:1/Fs:Nd/Fs; f=(0:length(t)-1)*Fs/length(t);  
>> tapN = 10; NanaI = 0.4*Nd+1; NanaF = 0.6*Nd;  
>> sigAF = fft(sigA);  
>> asigA = lpc(sigA(NanaI:NanaF),tapN); hsigA = freqz(1,asigA,f,Fs);  
>> plot(f, 20*log10(abs(hsigA))+15); axis([0 Fs/2 0 40]);
```

上記のasigA は母音 /a/ に対する声道フィルタの係数である.

母音 /a/ の声道フィルタ(係数asigA) に声帯音源波 xEnv を入力し, 合成音 /y/ を得る. 試聴するとともに, スペクトル, フォルマントを描く.

```
>> y = filter(1,asigA ,xEnv);  
>> yF = fft(y);  
>> sound(real(0.9*y/max(y)))  
>>  
>> plot(f, 20*log10(abs(yF))-25,'m--');axis([0 Fs/2 0 50]);  
>> title('Spectrum and formant of synthesized vowel /a/');  
>> xlabel('freq (Hz)'), ylabel('mag (dB relative)'),hold on  
>> plot(f, 20*log10(abs(hsigA))+15); hold off
```

演習課題3.3: 他の母音の合成音声を作るとともに, 基本周波数(ピッチ) を色々変えてみよ.

4.4 母音の合成II

2つのフォルマントを持つフィルタは次の式で表される 4次のフィルタ($n_a = 4$) で実現できる.

$$H_k = \frac{1}{(1 - \alpha_1 z^{-1})(1 - \alpha_1^* z^{-1})(1 - \alpha_2 z^{-1})(1 - \alpha_2^* z^{-1})}$$
$$\text{where } \alpha_1 = (1 - \varepsilon_1)e^{2\pi i f_1 / f_s}, \quad \alpha_2 = (1 - \varepsilon_2)e^{2\pi i f_2 / f_s}$$

ここで, $\varepsilon_1, \varepsilon_2$ は小さい数である. また, フォルマント周波数は f_1, f_2 である.

Matlab で 4 次フィルタを作り, 母音を合成する.

```
>> f1=700;f2=1300; e1=0.1; e2=0.07;
>> alph1 = (1-e1)*exp(2*pi*i*f1/Fs);
>> alph2 = (1-e2)*exp(2*pi*i*f2/Fs);
>> a4fil=conv(conv([1 -alph1],[1 -conj(alph1)]), ...
               conv([1 -alph2],[1 -conj(alph2)]));
>> h4filt=freqz(1, a4fil, f, Fs);
>> plot(f,20*log10(abs(h4filt))); axis([0 Fs/2 0 40]);
>>
>> y4fil = filter(1,a4fil,xEnv);
>> sound(0.9*y4fil/max(y4fil))
```

レポート課題(第3週)

1. 演習課題3.1-3.3について報告しよう。
2. 合成方法のI, II を用いて, 5母音を合成せよ. 基本周波数, フォルマント周波数, フォルマントの大きさなどをいろいろ変えてみよ. F_1 - F_2 の図(母音の地図) 上で色々な音を作り, 試聴せよ. 合成音の品質はどの程度か? 実音声と何が違うか? 波形, スペクトル, 音色等を比べよ。
3. 実験に収録した音声ファイルと合成した音声ファイル (wav形式で出力) は添付して提出すること。

【発展課題】 下記の方法によりPraat[5]を用いて5母音の合成音を作成し, 比較してみよう.

方法1 Praatの[Filter]機能によって5母音の合成音を作成する。

方法2 録音した母音音声をPraatに読み込むことで声帯音源波を抽出し (wavフィルとして出力), その声帯音源波をMATLAB (合成 I または合成 II) にインポートすることで合成音を作成する

参考文献

- [1] 小林一行, MATLAB ハンドブック, 秀和システム1999.
- [2] 芦野隆一, R'emi Vaillancourt, はやわかり MATLAB, 共立出版 1997.
- [3] 板橋秀一 編著, 音声工学, 森北出版
- [4] <http://www.speech.kth.se/wavesurfer/>
- [5] <http://www.fon.hum.uva.nl/praat/>

付録：Matlab 入門

1 起動と終了

1.1 起動

kterm から以下のコマンドを入力する[†].

```
% matlab
```

Matlab 用のデスクトップ環境が現れる. 入力を促すプロンプト (>>) が表示される.

```
< M A T L A B >
Copyright 1984-2001 The MathWorks, Inc.
Version 6.1.0.450 Release 12.1
May 18 2001

To get started, type one of these: helpwin, helpdesk, or demo.
For product information, visit www.mathworks.com.

>>
```

1.2 終了

Matlab の入力行に `exit` を入力する. 計算回数を一瞬表示した後, Matlab 用のウインドウが消える.

```
>> exit

260774 flops.
```

以後, Matlab が起動されている状態で説明を行う. 説明の中で行列とあった場合, 特に断りのない限り, 列ベクトルや行ベクトルも含む.

1.3 オンラインヘルプ

演算	Matlab での表現	備考
ヘルプ (コマンド行)	<code>help command-name</code>	
ヘルプ (別ウインドウ)	<code>helpwin command-name</code>	
ヘルプ (ブラウザ)	<code>helpdesk</code>	
コマンド/関数の検索	<code>lookfor string</code>	

[†] デスクトップ環境が必要ない人は `% matlab -nodesktop` で立ち上げるとよい. kterm 上にプロンプトが表示される.

コマンドの使い方やコマンド名が分からないとき、どんなコマンドがあるかを知りたいときのため、ヘルプや検索が用意されている。慣れれば、参考書を見ずにオンラインヘルプだけで計算を進めることができる。

`help` のあとにコマンド名を入力すると、コマンドの説明が表示される。

```
>> help exit
```

```
EXIT    Exit from MATLAB.  
EXIT terminates MATLAB.
```

`help` だけ入力すると、Matlab の項目一覧が示される。 `help` のあと項目を指定すると、その項目に関するコマンドの一覧が表示される。

```
>> help
```

```
HELP topics:
```

```
matlab/general      - General purpose commands.  
matlab/ops          - Operators and special characters.  
matlab/lang         - Programming language constructs.  
...
```

```
>> help general
```

```
General purpose commands.
```

```
MATLAB Toolbox   Version 5.2 18-Dec-1997
```

```
General information
```

```
help          - On-line help, display text at command line.  
helpwin       - On-line help, separate window for navigation.  
helpdesk      - Comprehensive hypertext documentation and trouble  
...
```

コマンドを検索する例を示す。FFT に関するコマンドを検索する。

```
>> lookfor fft
```

```
FFT Discrete Fourier transform.  
FFT2 Two-dimensional discrete Fourier Transform.  
FFTN N-dimensional discrete Fourier Transform.  
FFTSHIFT Shift DC component to center of spectrum.  
IFFT Inverse discrete Fourier transform.  
...
```

2 データ処理の基礎

2.1 数値演算

演算	Matlab での表現	備考
演算子	<code>+ - * / ^ ...</code>	
式の区切り	<code>, ;</code> 改行	
コマンドの実行	<code>command-name arg1 arg2...</code>	<i>ex.</i> <code>help demo</code>
関数の呼出し	<code>func-name(arg1, arg2,...)</code>	<i>ex.</i> <code>max(3,4)</code>

Matlab の入力行では, `^f`, `^b`, `^p`, `^n`, `^a`, `^e`, `^k` などの emacs ライクなコントロールコードを用いて柔軟な編集が出来る.

関数電卓のように計算が出来る.

```
>> (1+3.5*exp(-2.1))/4.9^1/2

ans =

    0.1458
```

変数を使える. 結果を表示したくないときには ; で区切る.

```
>> x=(1+3.5*exp(-2.1))/4.9^1/2;
>> x+1

ans =

    1.1458
```

始めから用意されている定数として, 虚数単位 (`i` または `j`), 円周率 (`pi`), 無限大 (`inf`), Not-a-Number(`NaN`), 直前の結果 (`ans`) などがある. 但し定数は変数としても使用することが出来る. (以下 `i` の使用方法を参照のこと)

```
>> pi

ans =

    3.1416

>> 10^1000

ans =

    Inf

>> 3+2*i

ans =
```

```
3.0000 + 2.0000i

>> i = 1;
>> 3+2*i

ans =

    5

>> i=sqrt(-1)

i =

    0 + 1.0000i
```

2.2 数学関数

演算	Matlab での表現	備考
平方根	y = sqrt(x)	$y = \sqrt{x}$
指数	y = exp(x)	$y = e^x$
対数	y = log10(x)	$y = \log_{10} x$
自然対数	y = log(x)	$y = \log_e x$
三角関数	y = sin(x), cos(x), tan(x), asin(x), acos(x), atan(x)	$y = \sin(x), \cos(x), \tan(x)$ $\arcsin(x), \arccos(x), \arctan(x)$
絶対値, 位相角	y = abs(x), angle(x)	$y = x , \arg(x)$
実部, 虚部	y = real(x), imag(x)	$y = \operatorname{Re}(x), \operatorname{Im}(x)$
四捨五入, 切捨て	y = round(x), fix(x)	

複素数の絶対値と位相角を求める。(一行で3つの処理を行う例。セミコロン(;)の付いていない後ろ2つの処理に対して、結果が表示される。)

```
>> a=1+1*i; abs(a), angle(a)*180/pi

ans =

    1.4142

ans =

    45
```

2.3 行列の操作

演算	Matlab での表現	備考
ベクトルの長さ	<code>n = length(v)</code>	
行列の行数	<code>m = size(a,1)</code>	
行列の列数	<code>n = size(a,2)</code>	
行列の大きさ	<code>[m n] = size(a)</code>	
数列の作成	<code>start:end</code> または <code>start:step:end</code>	<i>ex.</i> <code>0:1/10:5</code>
要素が 0 の行列の作成	<code>a = zeros(size)</code>	<code>zeros(4)</code> で
要素が 1 の行列の作成	<code>a = ones(size)</code>	4 × 4 の正方行列
単位行列の作成	<code>a = eye(size)</code>	<code>zeros(2,3)</code> で
ランダム行列の作成	<code>a = rand(size)</code>	2 行 3 列の行列

- 行列の作成例を示す。

列ベクトル (3 5 7) を変数 *a* に代入する。ベクトルや行列の各列は, [] の中に空白 (またはカンマ(,)) で区切る。

```
>> a = [3 5 7]

a =
```

```
    3    5    7
```

行ベクトル $\begin{pmatrix} 2 \\ 4 \\ 6 \end{pmatrix}$ を変数 b に代入する。ベクトルや行列の各行は、セミコロン (;) で区切る。

```
>> b=[2; 4; 6]
```

```
b =
```

```
2
4
6
```

2 行 3 列の行列作成。
各列の要素を空白で、各行の要素をセミコロン (;) で区切って [] の中に並べる。

```
>> A=[2 3 4; 5 6 7]
```

```
A =
```

```
2    3    4
5    6    7
```

要素がすべて 0 の 2×4 行列を作成する。

```
>> X=zeros(2,4)
```

```
X =
```

```
0    0    0    0
0    0    0    0
```

要素がすべて 1 の列ベクトルを作成する。

```
>> y=ones(1,5)
```

```
y =
```

```
1    1    1    1    1
```

0 から 1 までの乱数を要素とする 2×3 行列を作る。

```
>> rand(2,3)
```

```
ans =
```

```
0.9501    0.6068    0.8913
0.2311    0.4860    0.7621
```


1 から 5 までの数列と, 20 から 12 まで 2 ずつ減少する数列を作る.

```
>> x=1:5, y=20:-2:12

x =

     1     2     3     4     5

y =

    20    18    16    14    12
```

- 行列の長さ, 大きさを調べる.

行列 A , ベクトル v がそれぞれ次のように定義されているとき,

```
>> A=[2 3; 4 5; 6 7];
>> v=[4 -1];
```

行列 A の大きさを調べる.

```
>> size(A)

ans =

     3     2
```

結果を n と m の 2 つの変数で受け取ることも出来る. `size()` のように, Matlab には 2 つ以上の値を返す関数もある. この場合, 左辺には変数名を (`[ ]`) の中に並べる.

```
>> [n m]=size(A)

n =

     3

m =

     2
```

ベクトル v の長さを調べる.

```
>> length(v)
```

```
ans =
```

```
2
```

- 行列要素の参照.

以下のようにコロンの(:)や[]を使って, 行列要素を指定できる.

$A(i,j)$ i 行 j 列目

$A(:,j)$ j 列目

$A(i,:)$ i 行目

$A([2\ 4\ 5],j)$ 2, 4, 5 行 j 列目

$A(2:5,j)$ 2 ~ 5 行 j 列目

$A(2:5,:)$ 2 ~ 5 行目

$A(2:5,1:2)$ 2 ~ 5 行 1 ~ 2 列目

$A(2:2:7,j)$ 2, 4, 6 行 j 列目

行列 A の 2 ~ 5 行 2 列目を取り出す.

```
>> A(2:3,2)
```

```
ans =
```

```
5
```

```
7
```

行列 A の 2 と 1 行目の, 2 と 1 列目を取り出す.

```
>> A([2 1],[2 1])
```

```
ans =
```

```
5      4
```

```
3      2
```

行列 A の 2 行目の, 1,2 列目にベクトル v を代入する.

```
>> A(2,1:2) = v
```

```
A =
```

```
2      3
```

```
4     -1
```

```
6      7
```

2.4 行列と配列の演算

Matlab では行列と配列の演算を明確に区別している。行列の演算とは、数学的に定義された行列に対する演算である。また、配列の演算とは、単なるデータの並びに対する演算であり、各要素ごとにスカラー演算が行われる。

演算	Matlab での表現	備考
行列 (ベクトル) 演算	$+$ $-$ $*$ $/$ $^{\wedge}$	
配列演算	$+$ $-$ $.*$ $./$ $.^{\wedge}$	
行列式	<code>d=det(A)</code>	
転置共役	<code>A'</code>	ピリオド (.) に注意 複素行列の場合、共役となる。
転置	<code>A.'</code>	複素行列でも、共役にならない。
逆行列	<code>B=inv(A)</code>	
ランク	<code>n=rank(a)</code>	
固有値, 固有ベクトル	<code>[v l]=eig(A)</code>	$A \cdot v = lv$

行列のスカラー倍.

```
>> 3*A
```

```
ans =
```

```
6     9
12    -3
18    21
```

行列の転置.

```
>> A'
```

```
ans =
```

```
2     4     6
3    -1     7
```

配列同士の積演算.

u , v および演算結果は同じ長さの同じ種類のベクトルである*.

```
>> u = [2; 3];  
>> u.*v  
  
ans =  
  
      8      -3
```

行列としての積の演算例.

$n \times k$ 行列と $k \times m$ 行列の演算のみ可能. 特に $n = m = 1$ のときにはベクトルの内積となる.

```
>> u*v'  
  
ans =  
  
      5  
  
>> A*v'  
  
ans =  
  
      5  
     17  
     17
```

- 変数の管理

演算	Matlab での表現	備考
変数の一覧	<code>who</code>	
変数の詳細一覧	<code>whos</code>	
変数の消去	<code>clear var-name</code>	

*演算されるベクトルの種類が異なるとき, つまり, 一方が行ベクトル, 他方が列ベクトルのときには, どちらかのベクトルを転置することで配列演算可能となる.

`who` は変数の一覧を，`whos` は詳細一覧を表示する．

```
>> who

Your variables are:

A          ans          u
X          m            v

>> whos

Name      Size      Bytes  Class

A         3x2         48  double array
X         2x4         64  double array
ans       3x1         24  double array
m         1x1          8  double array
u         1x2         16  double array
v         1x2         16  double array

Grand total is 22 elements using 176 bytes
```

要らなくなった変数は `clear` で消去できる．
但し，引数なしで実行すると，全ての変数が消去されるので注意すること．

```
>> clear m u v
>> who

Your variables are:

A          X          ans
```

2.5 統計処理

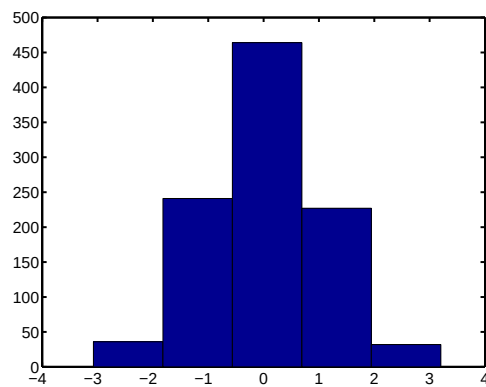
演算	Matlab での表現	備考
最大，最小	<code>y=max(x), min(x)</code>	n は分割数
総和，累積和，総積	<code>y=sum(x), cumsum(x), prod(x)</code>	
平均，分散，標準偏差	<code>y=mean(x), cov(x), std(x)</code>	
中央値，相関係数	<code>y=median(x), corrcoef(x)</code>	
正規分布	<code>y=randn(size)</code>	
ヒストグラム	<code>y=hist(data, n)</code> <code>[n x]=hist(data, n)</code>	

正規分布に従う，1000 個のランダムデータの作成．最大値，最小値，平均値の表示．

```
>> data=randn(1,1000);  
>> max(data), min(data), mean(data)  
  
ans =  
  
    3.2025  
  
ans =  
  
   -3.0737  
  
ans =  
  
    0.0455
```

ヒストグラムの表示．
左辺を指定すると，
データの頻度値，範囲
値が表示される．この
場合グラフはプロット
されない．

```
>> hist(data,5);
```



```
>> [n h]=hist(data,5)
```

```
n =
```

```
    36    241    464    227    32
```

```
h =
```

```
  -2.4461  -1.1909   0.0644   1.3196   2.5749
```

2.6 ファイル入出力

Matlab には C ライクなファイル入出力関数が定義されている.

演算	Matlab での表現	備考
オープン	<code>fid=fopen('file-name','mode')</code>	<code>fid</code> : ファイル識別子 <code>mode</code> 例: <code>r</code> , <code>w</code>
クローズ	<code>fclose(fid)</code>	
アスキーファイル用 読み込み 書き出し	<code>x=fscanf(fid,'format')</code> <code>fprintf(fid,'format',x)</code>	<code>format</code> 例: <code>%d</code> , <code>%f</code>
バイナリファイル用 読み込み 書き出し	<code>x=fread(fid,'precision')</code> <code>fwrite(fid,x,'precision')</code>	<code>precision</code> 例: <code>int16</code> <code>float</code>

10 個の整数からなる配列 `x` をアスキーファイル `afile.data` への書き出す. ファイル `afile.data` の内容を表示する*.

```
>> x=[1 1 2 2 3 3 4 5 6 7];
>> fout=fopen('afile.data','w');
>> fprintf(fout,'%d ',x); fprintf(fout,'\n');
>> fclose(fout);
>> !cat afile.data
1 1 2 2 3 3 4 5 6 7
```

アスキーファイル `afile.data` からデータを変数 `y` に読み込む.

```
>> fin=fopen('afile.data','r');
>> y=fscanf(fin,'%d');
>> fclose(fin);
>> y'

ans =

     1     1     2     2     3     3 ...
```

*表示にはシェルコマンドの `cat` を使っている. シェルにエスケープするための `!` に注意. `!` の後には任意の UNIX コマンドを実行できる.

10 個の整数からなる配列 `x` をバイナリファイル `bfile.data` への書き出す。UNIX のコマンド `od` で `bfile.data` の内容を表示する。

```
>> fout=fopen('bfile.data','w');
>> fwrite(fid,x,'int16');
>> fclose(fin);
>> !od -A n -t d2 bfile.data
      1      1      2      2      3      3 ...
```

バイナリファイル `bfile.data` からデータを変数 `y` に読み込む。

```
>> fin=fopen('bfile.data','r');
>> y=fread(fin,'int16');
>> fclose(fin);
>> y'

ans =

      1      1      2      2      3      3 ...
```

2.7 プログラミング

演算	Matlab での表現	備考
条件分岐	<code>if expr, states, end</code> <code>if expr, states, else states, end</code> <code>if expr, states, elseif expr, states, else states, end</code>	
繰り返し	<code>for var=expr, states, end</code> <code>while expr, states, end</code>	
ループの脱出	<code>break</code>	
関数の定義	<code>function</code>	
関数からの復帰	<code>return</code>	

`for` 文や `while` 文を使って、良く似た処理を連続して用いたり、`if` 文で、値の大小等によって処理を変えたりすることが出来る。条件判断には、`<`, `>`, `<=`, `>=`, `==`, `~=`, `&`, `|` などの表現を使う。

Matlab において `for`, `while`, `if` などの処理の流れをコントロールするコマンドの実行速度は、行列演算に比べ、かなり遅い。可能な限り、行列演算を使うことを心がけよ。

また、何度も使う処理をまとめて新しい関数を定義することが出来る。

`data` の中で 3 以上 5 より小さい要素の個数を数える.

```
>> data=[3 8 1 3 2 7 9 1 2 1];
>> c=0;
>> for n=1:length(data)
            if (data(n) >= 3) & (data(n) < 5)
                c=c+1;
            end
        end
>> c

c =

    2
```

上記の処理を行う関数を定義する. 以下の内容のファイルを用意する. ファイル名を `count3to5.m` とすれば, 関数名は `count3to5` となる.

1 行目: 関数の宣言.

2 から 5 行目: % で始まる行はコメント. 宣言の直後のコメントは `help` コマンドの表示として使われる.

7, 10 行目: コメント. `help` コマンドで表示されない.

8 行目: 引数の個数のチェック.

11 から 16 行目: 実際の処理.

```
% cat -n count3to5.m
1  function c=count3to5(data)
2  %count3to5:      counts number of elements equal to or larger
3  %                than 3 and less than 5.
4  %
5  %usage:          c=count3to5(data)
6
7  % check argument number
8  if nargin < 1, return, end
9
10 % counting
11 c=0;
12 for n=1:length(data)
13     if (data(n) >= 3) & (data(n) < 5)
14         c=c+1;
15     end
16 end
```

関数
count3to5
を使って、
同じ処理
を行う。

```
>> help count3to5

count3to5:      counts number of elements equal to or larger
                  than 3 and less than 5.

usage:          c=count3to5(data)

>> count3to5(data)

ans =

      2
```

3 グラフ表示

演算	Matlab での表現	備考
1次元データ描画	plot(y) plot(x,y) plot(x,y,'line type')	線種, 色の指定
2次元データ描画	surf(x,y,Z) contour(x,y,Z) imagesc(x,y,Z)	ZはZ=[1,2,3,...; 2,3,4,...;...]のような 2次元配列. surf(Z)等も あり.
タイトル表示 x 軸ラベル付け 描画ウィンドウの生成	title('string') xlabel('string') figure	ylabel も同様 複数のウィンドウを出した い時使う.
描画ウィンドウの消去 画像の消去 画像の保持 画像保持を解放 タイル表示 印刷	close clf hold on hold off subplot(n,m,l) print [-ddevice] filename	ウィンドウは消去されない. 重ね書きに用いる. device: ps, psc eps epsc 等

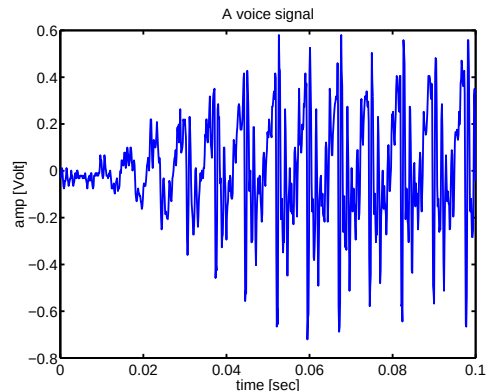
3.1 1次元データのグラフ表示

時間の配列を x 、信号を y としたとき、 x を横軸、 y を縦軸にプロットする。

x を省略すれば、 y の要素番号が横軸にプロットされる。

タイトルと縦横軸のラベルを付ける。

```
>> plot(x,y)
>> title('A voice signal');
>> xlabel('time [sec]');
>> ylabel('amp [Volt]');
```



`plot` の第3引数で、線種の指定が出来る。

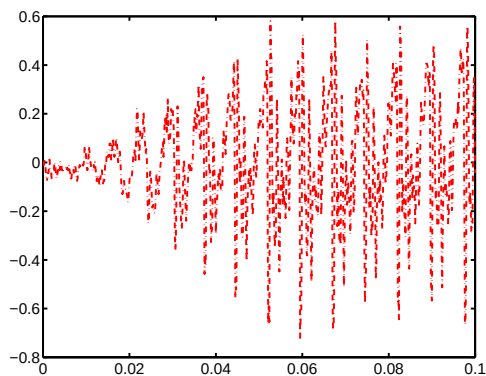
.	点	-	実線 (デフォルト)
o	丸	:	点線
x	×	--	鎖線
+	+	-.	一点鎖線
*	アスタリスク		etc.

以下の色指定が出来る。

y	黄色	g	緑
m	マゼンタ	b	青 (デフォルト)
c	シアン	w	白
r	赤	k	黒

線種と色の同時指定も出来る。

```
>> plot(x,y,':')
>> plot(x,y,'m')
>> plot(x,y,'-.r')
```



複数のグラフを別々のウィンドウで表示したいときには、`figure` コマンドで別の描画ウィンドウを生成した後、プロットする。

アクティブなウィンドウに描画されることに注意。マウスでクリックすることでウィンドウを選択できる。

必要のないウィンドウは `close` で消去できる。描画だけを消したい時は `clf` を使う。

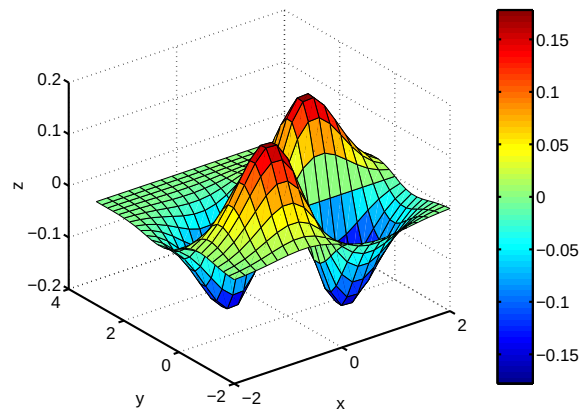
```
>> plot(x)
>> clf
>> plot(y)
>> figure
>> plot(x,y,'--')
>> close
```

3.2 2次元データのグラフ表示

サーフェース表示のグラフ描画.

`meshgrid(x,y)` で2変数関数を計算するための配列 `X`, `Y` を作ったのちサーフェース表示を行う.

```
>> x=-2:0.2:2;; y=-2:0.2:3;  
>> [X,Y] = meshgrid(x,y);  
>> Z = X.*Y.* exp(-X.^2 - Y.^2);  
>> surf(x,y,Z),xlabel('x')  
>> ylabel('y'),zlabel('z')  
>> colorbar
```

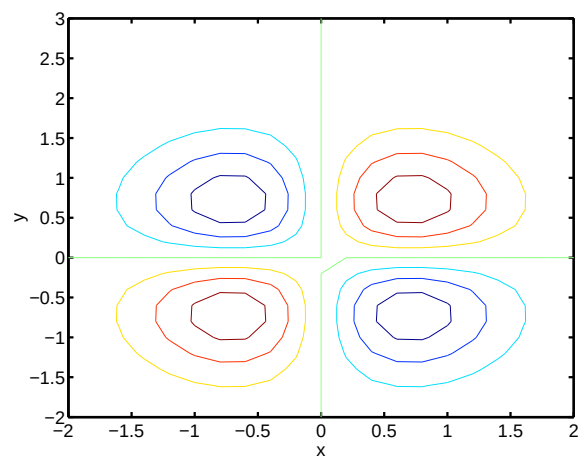


等高線表示のグラフ描画.

`contour(x,y,Z,N)` でレベル数 `N` を指定出来る. `N=20` ならレベル数 20 の等高線.

`contour(x,y,Z,V)` でレベル `V` を指定出来る. `V=[0 0.1 0.2]` ならレベル 0 と 0.1 と 0.2 の等高線.

```
>> contour(x,y,Z)  
>> xlabel('x'),ylabel('y')
```

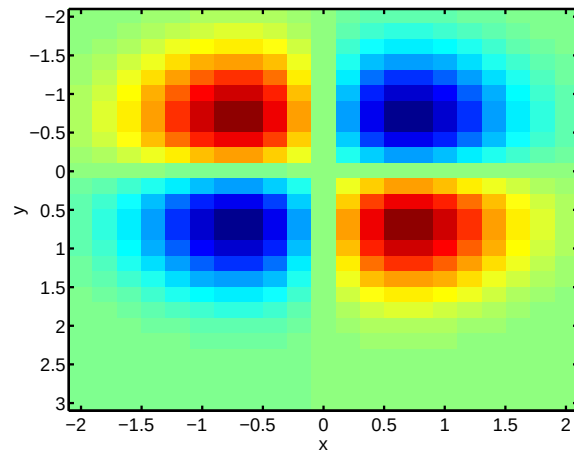


濃淡表示のグラフ描画.

y 軸に注意. 左上が原点となっている.

これは画像データを表示したとき, 上下反対にならないよう座標系を変えているためである.

```
>> imagesc(x,y,Z)
>> xlabel('x'),ylabel('y')
```

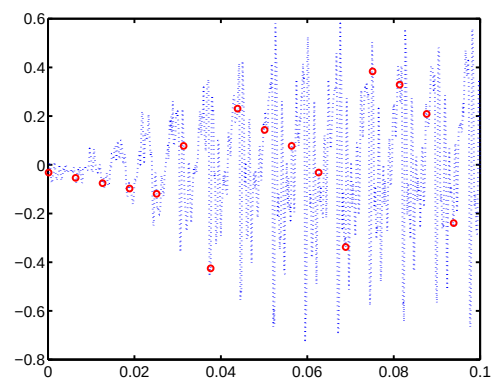


3.3 グラフの重ね書き, タイル表示

グラフの重ね書きする場合には, プロット関数の引数にデータを連ねればよい.

あるいは, hold on, hold off コマンドを使い, 描画の保持を制御する.

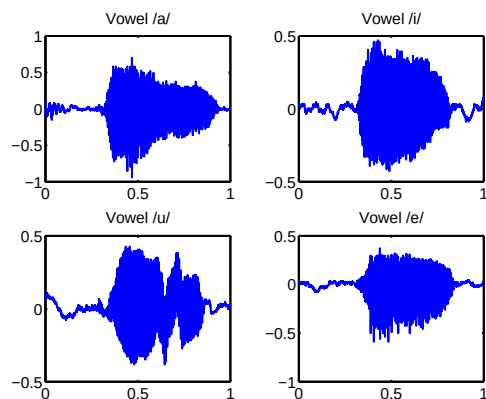
```
>> xspar=x(1:50:length(x));
>> yspar=y(1:50:length(y));
>>
>> plot(x,y,':',xspar,yspar,'or');
>>
>> plot(x,y,':'), hold on
>> plot(xspar,yspar,'or'), hold off
```



グラフのタイル表示の例.

`subplot(n,m,1)` で $n \times m$ 個のグラフを1つのウィンドウに表示出来る. 1 は, 左上から右下へ順序づけられたグラフの番号である.

```
>> subplot(2,2,1),plot(t,sigA);,title('Vowel /a/');
>> subplot(2,2,2),plot(t,sigI);,title('Vowel /i/');
>> subplot(2,2,3),plot(t,sigU);,title('Vowel /u/');
>> subplot(2,2,4),plot(t,sigE);,title('Vowel /e/');
```



3.4 グラフの出力

表示されたグラフをポストスクリプトファイルとしてファイルに出力させる. 出力ファイルは, 適当な Unix のコマンドで印刷出来る.

`-d` オプションで EPS(Encapsulated PostScript) 形式を指定すれば, $\text{\TeX}$ に取り込むことが出来る*. 単に `print` と打ち込めばデフォルトプリンタに出力される.

```
>> print -dps myplot.ps
>> print -depsc myplot.eps
>> print
```

4 信号処理

演算	Matlab での表現	備考
フーリエ変換	<code>y=fft(x)</code>	$y_k = \sum_{n=1}^N x_n e^{-2\pi i(k-1)(n-1)/N}$
逆フーリエ変換	<code>x=ifft(y)</code>	$x_n = \frac{1}{N} \sum_{k=1}^N y_k e^{2\pi i(n-1)(k-1)/N}$
畳み込み	<code>c=conv(a,b)</code>	$c_n = \sum_{k=1}^N a_k b_{n-k}$
逆畳み込み	<code>[q r]=deconv(b,a)</code>	$b = \text{conv}(a, q) + r$
デジタルフィルタ	<code>y=filter(b,a,x)</code>	$\text{conv}(a, y) = \text{conv}(b, x)$
スペクトログラム	<code>specgram(x)</code>	

*いくつか方法があるが, 一つの方法では, プリアンブルで `\usepackage{graphicx}` の指定をした後, 文中で `\includegraphics[width=2.5in]{myfile.eps}` などとすればよい.

4.1 フーリエ解析

アナログ信号 $x(t)$ があったとき、適当な条件のもと[†]で、

$$\begin{aligned}x(t) &= \frac{1}{2\pi} \int_{-\infty}^{\infty} d\omega y(\omega) e^{i\omega t}, \\y(\omega) &= \int_{-\infty}^{\infty} dt x(t) e^{-i\omega t},\end{aligned}$$

が成り立つ。第1の式は、信号 $x(t)$ が様々な(角)周波数 $\omega = 2\pi f$ を持つ正弦振動 $e^{i\omega t}$ の重ね合わせとして書きあらわせることを示している。周波数 ω で振動する成分の(複素)振幅は $y(\omega)$ である[‡]。第2の式は、振幅 $y(\omega)$ が信号 $x(t)$ からどのように計算されるかを示している。信号 $x(t)$ から振幅 $y(\omega)$ を求める操作(第2式)をフーリエ変換、 $y(\omega)$ から信号 $x(t)$ を再現する操作(第1式)を逆フーリエ変換と呼ぶ。 $y(\omega)$ のことを、周波数 ω の周波数成分と呼ぶことは自然である。また、 $y(\omega)$ を(周波数)スペクトルと呼んだりする。

デジタル信号 $x_1, x_2, \dots, x_N$ があったとき、以下のような、アナログ信号の場合と類似の式が成立する。

$$\begin{aligned}x_n &= \frac{1}{N} \sum_{k=1}^N y_k e^{2\pi i \frac{(n-1)(k-1)}{N}}, \\y_k &= \sum_{n=1}^N x_n e^{-2\pi i \frac{(n-1)(k-1)}{N}}.\end{aligned}$$

信号 x_n から(複素)振幅 y_k を求める操作(第2式)を離散フーリエ変換(DFT, Discrete Fourier Transform)、振幅 y_k から信号 x_n を再現する操作(第1式)を離散逆フーリエ変換と呼ぶ。

アナログ信号からデジタル信号を得る時、1秒間に何回の割でサンプリングしたかを表すのがサンプリング周波数 f_s である。例えば、 f_s が 8000 Hz であれば、デジタル信号のサンプリング間隔は $1/8000$ 秒である。デジタル信号では $-f_s/2$ から $f_s/2$ までの周波数成分しか表現されない。 y_1 から $y_{[N/2]}$ までの振幅が 0 から $f_s/2$ の周波数成分、 $y_{[N/2]+1}$ から y_N までの振幅が $-f_s/2$ から $-f_s/2 + f_s/N$ までの周波数成分を表す。つまり、正の周波数成分は数列 y_k の前半に、負の周波数成分は y_k の後半にあることに注意すること。

離散フーリエ変換を高速に計算するアルゴリズムとして高速フーリエ変換(FFT, Fast Fourier Transform)がある。Matlab には高速フーリエ変換が実装されており、その関数名は `fft` である。

- フーリエ変換を使った信号処理の例。

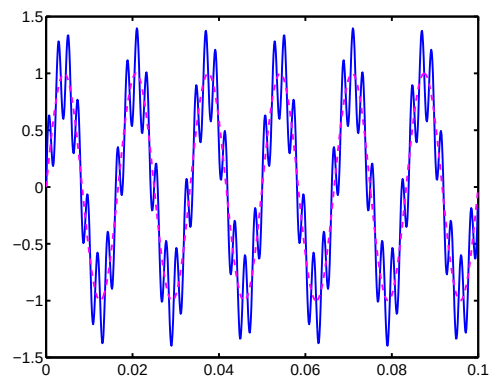
[†]例えば、 $\left| \int_{-\infty}^{\infty} dt x(t) \right| < \infty$ 。

[‡] $x(t)$ は、普通、実数であるため、振幅 $y(\omega)$ は、 $y^*(\omega) = y(-\omega)$ の性質を満たす。

60 Hz と 440 Hz の正弦波 x_1 , x_2 を重ね合わせて、サンプリング周波数 8000 Hz で離散化された 0.1 秒間の信号 x をつくる。 t は時間を表す配列である。

x を実線で、 x_1 を破線でグラフ表示する。 x_1 で示される遅い振動に、速い振動が重畳されているのがわかる。以下では、早い振動を除去する処理を行う。

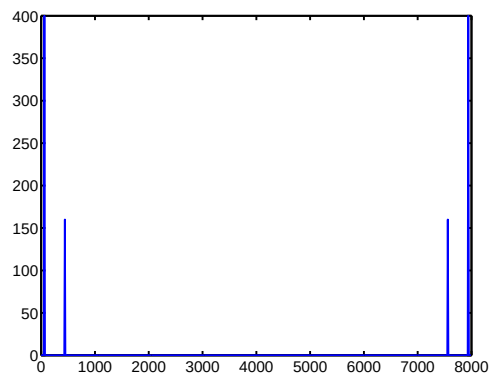
```
>> fs=8000; dt=1/8000; t=0:dt:0.1-dt;
>> f1=60; f2=440;
>> x1=1.0*sin(2*pi*f1*t);
>> x2=0.4*sin(2*pi*f2*t);
>> x=x1+x2;
>> plot(t,x,t,x1,'m--')
```



処理の前に、 x のスペクトルを観察しよう。周波数を表す配列 f を作った後、 x をフーリエ変換する。

周波数成分 (複素振幅) xF の絶対値を表示すると、60 Hz と 440 Hz にピークが現れる。7940 Hz と 7560 Hz にもピークが現れるがこれは各々 -60 Hz, -440 Hz に対応する成分である。

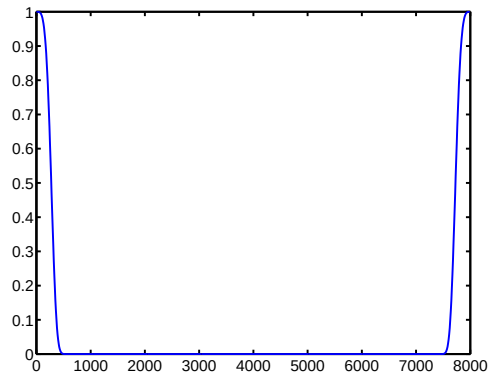
```
>> f=(0:length(t)-1)*fs/length(t);
>> xF=fft(x);
>> plot(f,abs(xF));
```



300 Hz にカットオフをもつ適当なローパスフィルタ関数 `lpassF` を定義する. `lpassF` の右辺第1項は, 正の周波数成分に対するカットオフ, 第2項は, 負の周波数成分に対するカットオフである.

`lpassF` のように長い式は ... をつかって2行に書き分けることも出来る.

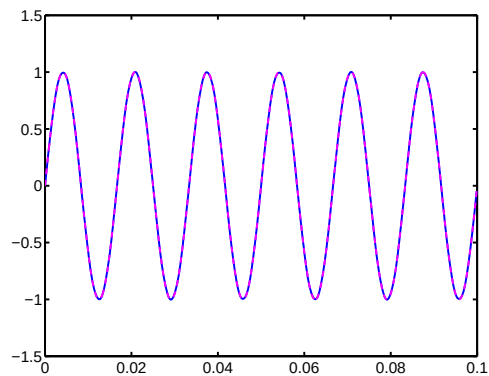
```
>> fc=300;
>> lpassF=exp(-(f./fc).^4)+...
exp(-((fs-f)./fc).^4);
>> plot(f,lpassF);
```



信号 `x` の周波数領域表示 `xF` にフィルタ関数 `lpassF` を掛け合わせたものを逆フーリエ変換し, 低い周波数成分のみをもつ時間領域信号 `xlpass` を得る

`xlpass` を表示する, 元の `x1` は破線で表示されているが重なってしまっている. つまり, 速い振動はほぼ完全にに取り除かれている.

```
>> xlpass=real(ifft(xF.*lpassF));
>> plot(t,xlpass,t,x1,'m--')
```



4.2 フィルタ

ある線形システムに, デジタル信号 $x_1, x_2, \dots, x_N$ が入力された時, 新しいデジタル信号 $y_1, y_2, \dots, y_N$ が出力されたとする. 線形システムは, 次の式を満たす2組のフィルタ係数 $a_1 (=1), a_2, \dots, a_{n_a+1}$ および $b_1, b_2, \dots, b_{n_b+1}$ で特徴づけられる.

$$y_n + a_2 y_{n-1} + \dots + a_{n_a+1} y_{n-n_a} = b_1 x_n + b_2 x_{n-1} + \dots + b_{n_b+1} x_{n-n_b}.$$

上式の左辺は、信号 y_n とフィルタ係数 a_n の畳み込み、右辺は、信号 x_n とフィルタ係数 b_n の畳み込みに他ならない。すなわち、

$$\sum_{m=1}^{n_a+1} y_{n-m+1} a_m = \sum_{m=1}^{n_b+1} x_{n-m+1} b_m, \quad \text{for } n=1, \dots, N.$$

と書ける。

線形システム、つまり、フィルタ係数 a_n, b_n が既知であり、入力時系列 x_n がシステムに与えられた場合、出力時系列 y_n を求める方法をアルゴリズム的に書けば、

$$\begin{aligned} y_1 &= b_1 x_1, \\ y_2 &= b_1 x_2 + b_2 x_1 - a_2 y_1, \\ y_3 &= b_1 x_3 + b_2 x_2 + b_3 x_1 - a_2 y_2 - a_3 y_1, \\ &\dots \end{aligned}$$

となる。もし $n_b = 0$ のとき、つまり b_1 以外の b_n がゼロのとき、フィルタは IIR (Infinite Impulse Response)、または、全極型フィルタと呼ばれる。もし $n_a = 0$ のとき、つまり a_1 以外の a_n がゼロのとき、フィルタは FIR (Finite Impulse Response)、または、全ゼロ型フィルタと呼ばれる。

フィルタリングにより、信号のスペクトル (周波数特性) が変化する。入力時系列 x_n のスペクトルを X_k とすれば、出力時系列 y_n のスペクトル Y_k は次の式で与えられる。

$$Y_k = H_k X_k,$$

ここで、 H_k はフィルタのスペクトル (周波数特性) であり、

$$H_k = \frac{b_1 + b_2 z^{-1} + \dots + b_{n_b+1} z^{-n_b}}{a_1 + a_2 z^{-1} + \dots + a_{n_a+1} z^{-n_a}}$$

と表される。ただし、

$$z = e^{2\pi i(k-1)/N}$$

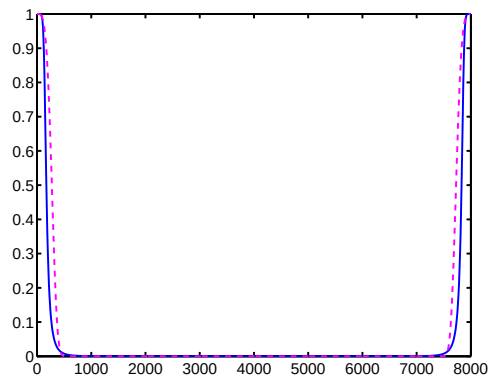
である。上式は、入力信号のスペクトルに H_k を掛算すれば、出力信号のスペクトルが求まることを示している。

- フィルタを使った信号処理の例。

時間領域でフィルタを設計することにより，前節と同じ信号 x に対し，ローパスフィルタリングを行う。

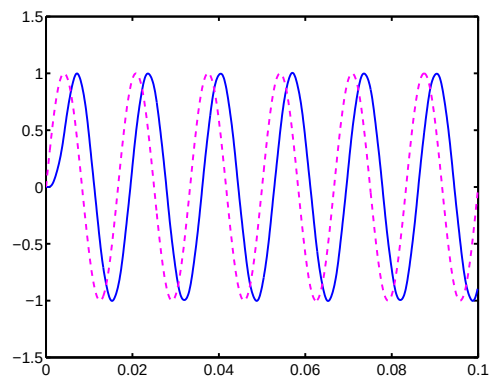
代表的なローパスフィルタに Butterworth フィルタがある．カットオフ周波数が $f_c=300$ Hz となるような Butterworth フィルタを，Matlab 関数 `butter` を用いて作り，フィルタ係数 b ， a を求める．ここでは，フィルタの次数を 4 とした．推定されたフィルタの周波数特性は，関数 `freqz` で計算できる．

```
>> [b a]=butter(4,fc/fs);  
>> plot(f,abs(freqz(b,a,f,fs))), hold on  
>> plot(f,abs(lpassF),'m--'); hold off
```



関数 `filter` を用いて，フィルタリングを行う．速い振動が取り除かれている．フィルタリングされた信号の位相が，フィルタの次数分だけ，遅れていることに注意．

```
>> y=filter(b,a,x);  
>> plot(t,y,t,x1,'m--');
```



フィルタの定義によると，フィルタリングされた信号 y とフィルタ係数 a の畳み込みは，原信号 x とフィルタ係数 b の畳み込みと一致する．この事実を確認する．

```
>> lhs=conv(y,a);lhs=lhs(1:length(t));  
>> rhs=conv(x,b);rhs=rhs(1:length(t));  
>> plot(t,lhs,t,rhs,'m--');
```

